FYND HACKATHON 2026 · CHALLENGE C14

FollowThru

Continuation Prompt Playbook

Chase what's late. Never what's done.

> **Starts from:** branch `full-integration`, commit `cad765d`
>
> **Repository:** `D:\followthru` · github.com/shivv-14/followthru
>
> **Audited:** 17 Oct 2026
>
> **Contents:** 20 continuation prompts (C01-C20) and 6 operational prompts
>
> **Workflow:** single laptop, Claude Code in VS Code

Every prompt is a complete copy-paste block. Run one prompt per new Claude Code chat.

Contents

1. How to use this playbook 3
2. Verified starting point 4
3. Original prompts P01-P27 mapped to the repository 6
4. Continuation order and dependencies 8
5. Continuation prompts 10
   - C01 · Project memory file for the single-laptop workflow 11
   - C02 · Challenge fixture intake, fixture notes and adapter 14
   - C03 · Make the engine pass the labelled commitments 17
   - C04 · Deadline parser: explicit timezones, relative and numeric dates 20
   - C05 · Message signals: progress, not done, blockers, reopen, extractor seam 23
   - C06 · Reminder wording by response state, subjects and escalation recipient 26
   - C07 · Product change record: starting state, scope and final evidence 29
   - C08 · Draft review actions: reject, edit and clarification lane 32
   - C09 · Role-based navigation and member workspace gaps 35
   - C10 · Backup, restore, evidence pack and audit filters 38
   - C11 · REST API completion and API documentation 41
   - C12 · PWA completion: icons, manifest, install, offline and update 44
   - C13 · Quality pass: code splitting, error boundaries, accessibility, layout, drive-letter test bug 47
   - C14 · Commitment detail: plain-English Why, time travel and thread view 50
   - C15 · Landing page scroll story 53
   - C16 · Optional AI-assisted extraction with a labelled rules fallback 56
   - C17 · Screenshot evidence and project documentation 59
   - C18 · Final QA and release check 62
   - C19 · Merge into main and deploy on Vercel 65
   - C20 · Android APK with PWABuilder and asset links 68
6. Reusable operational prompts 71
   - CONTINUE · Resume after a new Claude Code session 72
   - FIX · Reproduce and repair a specific bug 73
   - SCORECARD · Run the evaluation and report actual results 74
   - INTEGRATION CHECK · Confirm the UI is wired to the real engine and store 75
   - RELEASE CHECK · Verify the full acceptance checklist before a release 76
   - BLOCKER REPORT · List missing external inputs and keep independent work moving 77
7. What Claude does and what you do 78
8. Final release and submission checklist 79
9. Audit notes 80

1. How to use this playbook

This playbook continues FollowThru from the state the repository is in today. It does not restart the project and it skips work that is already built and verified.

1. **One prompt per chat.** Open a new Claude Code chat (the + button), paste one whole grey block, and let it finish. Each block is self-contained, so nothing has to be pieced together from other prompts.
2. **Run C01 first.** It adds `CLAUDE.md`, which every later session reads automatically.
3. **Follow the order in section 4.** Essential prompts come first. Recommended and optional prompts can be skipped without breaking anything.
4. **Fixture-dependent prompts.** C02 and C03 need the real challenge file at `data/fixture.json`. Until you have it, skip them and continue with C04. Run them as soon as the file arrives, then re-run C07 step 3 and C18.
5. **Do the "You check" list** after every prompt. If something is wrong, use the FIX prompt in section 6 before moving on.
6. **After a limit or a crash,** use the CONTINUE prompt with the last prompt id you ran.
7. **main is protected.** Claude merges into `main` only when you write exactly *merge into main*. Deploying, the APK and the GitHub release need your accounts. Section 7 lists who does what.

Priority labels

Label	Meaning
Essential	Needed for the brief, the scoring evidence or a reliable demo.
Blocked	Waits for an input you must supply (the challenge fixture).
Recommended	Strong value for judges, safe to skip if time is short.
Optional	Enhancement from the original plan; cut first.

Rules every prompt enforces

Each block starts with the same project preamble: read the docs, inspect `git status`, keep uncommitted work, use the real scripts, keep the engine pure and the contract additive, make small lowercase commits with no AI mentions or co-author lines, never force-push or run destructive git commands, and report only verified results.

2. Verified starting point

Everything here was checked on 17 Oct 2026 by running the commands in this repository, not taken from earlier notes.

Item	Verified state
Working branch	`full-integration` at `cad765d` ("document integrated project and launch steps"). Identical on GitHub.
main	GitHub `main` = `3fd93f3` (frontend phase 1 merged, PR #4). The local `main` ref is stale at `9f400ad`. `full-integration` contains all of `main`.
Former tracks	`back-a-p0`, `back-a-p1`, `ui-b-p0` and `ui-b-p1` are fully contained in `full-integration`. Nothing is left to merge from them.
Uncommitted work	Only `docs/TRACK_A_BACKEND_PROMPT.md` (untracked, yours, left alone).
Scripts	dev, build, preview, test, test:watch, typecheck, evidence, e2e
npm run typecheck	Pass (exit 0)
npm test	148 tests in 11 files pass when run from `D:↑ollowthru`. **Reproducible failure:** started from a lowercase drive path (`d:/followthru`) all 11 suites fail with "TypeError: Cannot read properties of undefined (reading 'config')" and "no tests". VS Code sessions sometimes start in `d:↑ollowthru`, which caused 3 failed runs during this audit. C13 fixes it.
npm run build	Pass. Main JS 658.69 kB (209.19 kB gzip); 3D scene in a separate lazy chunk; PWA precache 18 entries.
npm run e2e	4/4 passed in the installed Chrome: coordinator journey, member view, 360 px width, API routes.
npm run evidence	Without the fixture it refuses and explains how to add it. With `--demo` it writes a scorecard labelled synthetic: check 1 not scored, checks 2 and 3 pass.
Challenge fixture	**Absent.** `data/` holds only README.md.
CLAUDE.md	**Absent** from this repository.
Earlier status PDF	The "project status and continuation guide PDF" mentioned in the request was **not found** anywhere on this laptop. This playbook does not rely on it.

Complete and verified (excluded from the continuation prompts)

- Project setup and tooling; strict TypeScript; Vitest, Testing Library, fake-indexeddb, Playwright with the installed Chrome.
- Shared contract `src/engine/types.ts` with export lock test; Zod validation with stable issue codes; CSV and JSON import and export with formula-injection escaping.
- Pure engine: deadline parsing in the speaker's timezone with DST rules and tentative hedges; reply, thread and cross-thread linking; accepted delegation; authority rules; review flags; response state; one versioned draft per commitment; suppression of closed work; clarify drafts; staleness; approval freshness check; recipient warnings; follow-up flows; permissions; scoring.
- Store: Dexie tables, audited repository, live hooks, SHA-256 evidence uploads, saved runs, reset.
- Screens on real engine data: setup with validation, profiles, dashboard, commitments with URL filters and exports, detail with evidence timeline and role actions, queue with recipient confirmation and freshness

blockers, flows editor, team (assign, submissions, blockers), evidence (scorecard, exports, uploads, audit, settings).

- API `/api/health` and `/api/process`, also served by `npm run dev`; `npm run evidence` with hashed outputs; offline PWA build (basic).

Implemented but not tested enough

- Flow editor, CSV import in Setup, evidence uploads in the browser, the settings toggles and reset: only partly covered by component or browser tests. C08 to C13 add coverage.
- Every engine rule has only been checked on synthetic conversations, never on the labelled challenge data (C02, C03).
- Offline behaviour and installability have never been tested (C12).

Blockers and external inputs

Input	Blocks	Who
Challenge fixture `Starter_Data/C14/fixture.json`	C02, C03, the 10-point owner/deadline/status score, real evidence	You or the organisers
Decision whether to commit the fixture	Deployed fixture loading and the /api/evidence source	You (check the challenge terms)
Vercel account	C19 deployment and live URL	You
PWABuilder output, Android phone	C20 APK	You
Gemini API key	C16 (optional)	You
Business case, pitch deck, demo video, teammate names, real growth-experiment rows	Submission checklist	You and your team
Official deadline and demo format	Planning	Organisers (not stated in the brief)

3. Original prompts P01-P27 mapped to the repository

Each original prompt was checked against the code and tests, not just against commit messages. "Design differs" means the goal is met with a different but equivalent structure.

ID	Original task	Status	Evidence in the repository	Continues in
P01	Project setup	Done and verified	Scripts dev, build, preview, test, typecheck present; build and tests pass.	-
P02	Project memory (CLAUDE.md)	Not implemented	No CLAUDE.md in D:\followthru (only in the abandoned scaffold).	C01
P03	Fixture notes	Blocked	Fixture absent; docs/fixture-notes.md is a template with the canonical format.	C02
P04	Contract, validation, loaders	Done (design differs)	types.ts, Zod schemas, 16 issue codes, CSV/JSON with formula escaping; 32 tests. Real-fixture adapter pending.	C02
P04b	Two-laptop rules	Obsolete	Single-laptop workflow; replaced by CLAUDE.md section.	C01
P05	Deadlines and timezones	Partial	Core parser, DST rules, hedges; 33 tests. Missing explicit zones (5pm IST), in N days, next week, numeric dates, some hedges.	C04
P06	Commitment extraction	Partial	Request, commit, accept, done, cancel, defer, reschedule, delegate, decline signals. Missing not-done, blocker, progress signals and an extractor interface.	C05
P07	History resolver	Mostly done	Reply, thread and cross-thread linking, accepted delegation, authority rules, review flags, response state. Missing reopen; real labels unverified.	C05, C03
P08	Reminder draft queue	Partial	One draft per commitment, versioning, suppression, clarify drafts, idempotent re-run; tested. Wording ignores response state; no subjects.	C06
P09	Evaluation and starting state	Partial	evaluate.ts and npm run evidence with hashes. No --label, latest copy, test report, git hash, suppressions CSV or starting-state run.	C07
P10	Stale suppression and recipient review	Partial	Engine, approval check, recipient warnings, confirmation and UI done and tested. Missing product-change docs, flag-off baseline, starting and final evidence.	C07, C08
P11	Follow-up flows	Mostly done	Default and custom flows, team precedence, working-days, editor, preview, validation. Missing escalation recipient and quiet-hours shift.	C06
P12	Task management	Mostly done	Person and team assignment, progress, blockers, submissions, approve or send back, coordinator decisions, permissions. Missing member 'need more time'.	C09
P13	Storage, audit, evidence files	Mostly done	Dexie tables, audited repo, SHA-256 uploads, runs, reset, live hooks; store tests. Missing backup export and restore.	C10
P14	REST API	Partial	/api/health and /api/process with limits, error shape, tests, dev-server plugin, vercel.json. Missing /api/evidence, CSV input, CORS, docs/api.md.	C11
P15	Design system and 3D foundation	Done (adequate)	Tokens, primitives, dialog, tabs, states, style guide, lazy 3D stage with 2D fallback.	-
P15b	Mock engine data	Superseded	Removed on purpose; every screen uses the real engine. Demo input is synthetic and labelled.	-
P16	Landing scroll story	Not implemented	Minimal hero over a one-mesh 3D stage.	C15 (optional)

P17	App shell, profiles, navigation	Partial	Setup import with validation, profiles from data, shell, clock, search. Missing role-based navigation and guards; PIN and command palette not built.	C09
P18	Coordinator dashboard	Done	KPIs, attention list, team health, 7-day strip, URL filters, CSV/JSON export from live engine data. 3D ring not built (2D bars).	-
P19	Commitment detail	Partial	Evidence timeline with rules and quotes, linked messages, draft card, role actions. Missing plain-English why, time travel, thread view.	C14
P20	Queue, recipient review, flow editor	Partial	Lanes, freshness blockers, confirm, approve, versions, suppression reasons, flow editor. Missing reject, edit, clarification lane.	C08
P21	Team pages	Mostly done	Assign form, submissions inbox, blockers, member list. Missing people grid with local times and extension requests.	C09
P22	Evidence centre, data, audit	Mostly done	Scorecard, runs, exports, uploads with hashes, audit log, settings, reset. Missing evidence pack, audit filters, restore, audited delete.	C10
P23	Optional AI mode	Not implemented	Flag exists and is labelled Rules (fallback).	C16 (optional)
P24	PWA	Partial	Manifest and service worker precache. Only an SVG icon; no install button, offline banner, update prompt; offline untested.	C12
P25	Polish pass	Partial	360 px dashboard verified, reduced motion in 3D stage. No route splitting, error boundaries or automated accessibility checks.	C13
P26	Screenshots and docs	Partial	README rewritten. No screenshot script or evidence folder; no architecture, pilot, growth or business docs.	C17
P27	Final QA	Not done	-	C18
MERGE	Merge ui into main	Done differently	Both tracks combined on full-integration (merge commit 00f2493).	-
Deploy	Vercel	Not done	Needs your account.	C19
APK	PWABuilder	Not done	Needs C12, a live URL and your phone.	C20

4. Continuation order and dependencies

Run the prompts top to bottom. Blocked prompts can be skipped and run later; recommended and optional prompts can be skipped. Engine changes (C04 to C06) come before evidence (C07), evidence before the queue and screen work that relies on it, PWA before the APK, and the release check before merging and deploying.

ID	Title	Priority	Continues	Needs first
C01	Project memory file for the single-laptop workflow	Essential	P02, P04b	None. C01 should run first because every later prompt reads CLAUDE.md.
C02	Challenge fixture intake, fixture notes and adapter	Blocked until you provide the fixture	P03, adapter part of P04	Needs C01. Needs the real fixture file from you. If you do not have it yet, skip to C04 and return here as soon as you get it.
C03	Make the engine pass the labelled commitments	Blocked until C02	Real-data verification of P06, P07, P08, P09	Needs C02. Re-run this prompt after C04 and C05 if the fixture arrives before them, because parser and signal changes can shift results.
C04	Deadline parser: explicit timezones, relative and numeric dates	Essential	Remaining part of P05	Needs C01. Independent of the fixture. If C03 already ran, re-run C03 afterwards.
C05	Message signals: progress, not done, blockers, reopen, extractor seam	Essential	Remaining parts of P06 and P07	Needs C01. Re-run C03 afterwards if the fixture is available.
C06	Reminder wording by response state, subjects and escalation recipient	Essential	Remaining parts of P08 and P11	Needs C05 (uses its not_done and progress events).
C07	Product change record: starting state, scope and final evidence	Essential	Remaining parts of P09 and P10	Needs C06. Re-run step 3 after C03 when the fixture is available.
C08	Draft review actions: reject, edit and clarification lane	Essential	Remaining part of P20 (queue)	Needs C06 (subjects shown on cards) and C07 (flag-aware approval).
C09	Role-based navigation and member workspace gaps	Essential	Remaining parts of P12, P17 and P21	Needs C08.
C10	Backup, restore, evidence pack and audit filters	Essential	Remaining parts of P13 and P22	Needs C09.
C11	REST API completion and API documentation	Essential	Remaining part of P14	Needs C07 (suppressions CSV). Independent of UI prompts.
C12	PWA completion: icons, manifest, install, offline and update	Essential (required for the APK)	Remaining part of P24	Needs C11 (vercel.json and API settled). Required before C20.
C13	Quality pass: code splitting, error boundaries, accessibility, layout, drive-letter test bug	Essential	P25	Needs C12. Run after the feature prompts so it checks the final screens.
C14	Commitment detail: plain-English Why, time travel and thread view	Recommended	Remaining part of P19	Needs C13 (lazy routes in place). Optional to skip if time is short.
C15	Landing page scroll story	Optional enhancement (cut first)	P16	Needs C13. Safe to skip entirely.
C16	Optional AI-assisted extraction with a labelled rules fallback	Optional (needs your own API key)	P23	Needs C05 and C10. Skip if you have no key or no time.
C17	Screenshot evidence and project documentation	Essential	P26	Needs C13 (and C14/C15 if you ran them, so screenshots show the final screens).

C18	Final QA and release check	Essential	P27	Needs all essential prompts that you chose to run (at least C01, C04 to C13 and C17).
C19	Merge into main and deploy on Vercel	Essential (needs your authorisation and Vercel account)	MERGE prompt and 'Deploy on Vercel' page	Needs C18 with no unexplained FAIL. Part B needs your exact phrase; Part C needs your Vercel URL.
C20	Android APK with PWABuilder and asset links	Optional or required by your submission (needs your accounts and phone)	'Android APK with PWABuilder' page	Needs C12 and C19 (live HTTPS URL).

If the fixture arrives late

Run C02, then C03, then re-run C07 step 3 (starting state and final evidence) and C18 (release check). Parser and signal changes from C04 and C05 are designed to keep synthetic results unchanged, so the order is safe either way.

5. Continuation prompts

Each prompt below has a short summary, the full copy-paste block, a "You check" list and the dependency note. Copy everything inside the grey block.

C01 Project memory file for the single-laptop workflow

Essential Continues: P02, P04b

Goal. Give every new Claude Code session the same rules, architecture map and commands by adding the missing CLAUDE.md, rewritten for the current single-laptop project.

Current context. CLAUDE.md does not exist in D:\followthru. An older version exists only in the abandoned scaffold at C:\Users\shiva\projects\followthru\CLAUDE.md and still describes two laptops and folders that changed. The real architecture, scripts and rules are listed in README.md and docs/integration-status.md.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C01 - Project memory file for the single-laptop workflow

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.
```

```
GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Give every new Claude Code session the same rules, architecture map and commands by adding the missing CLAUDE.md,
rewritten for the current single-laptop project.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
CLAUDE.md does not exist in D:\followthru. An older version exists only in the abandoned scaffold at
C:\Users\shiva\projects\followthru\CLAUDE.md and still describes two laptops and folders that changed. The real
architecture, scripts and rules are listed in README.md and docs/integration-status.md.

TASK: create CLAUDE.md in the repository root so every future session
starts with the correct rules. Documentation only, no code changes.

1. Read C:\Users\shiva\projects\followthru\CLAUDE.md for its wording on
   product rules and design language. Do NOT copy its two-laptop section,
   its old folder list (src/state, src/lib) or anything not true today.
2. Write CLAUDE.md with these sections, each short and factual:
   a. What we are building (two roles: coordinator and member, demo
      profiles, shadow mode, tagline).
   b. Scoring from the brief: 10 points owner/deadline/status for ten
      labelled commitments with evidence; 10 points zero reminders for
      completed, cancelled or deferred work; 5 points one action one draft
      and deadline updates change the same draft. Edge cases: maybe Friday,
      timezones, delegated owner, completion in another thread, no response
      versus not completed. Product change: stale-message suppression and
      recipient review; keep starting state, negotiated scope, final
      evidence.
   c. Golden engine rules (the 8 rules: silence never changes status;
      only owner, requester or coordinator or an authorised human decision
      changes status; ambiguous evidence -> needsReview, never a chase;
      now is always passed in; UTC storage, phrases read in the speaker
      timezone; every decision keeps evidence with message id, thread id,
      time, quote and rule; never hardcode fixture labels in the engine;
      one draft per commitment, id d_<commitmentId>, versioned).
   d. Single-laptop workflow: branch full-integration, push to
      origin full-integration, main only after the exact phrase
      "merge into main", no force push, no destructive git commands.
   e. Architecture map: list the real folders and key files with one line
      each (src/engine/*.ts, src/schema/*.ts, src/store/*.ts(x),
      src/pages/*.tsx, src/components/{ui,shell,domain}, src/three,
      src/styles, src/mocks/demoDataset.json, api/_lib/handlers.ts,
      api/health.ts, api/process.ts, scripts/evidence.ts, tests/, e2e/,
      vite.config.ts with the dev API plugin and PWA, vercel.json).
      Run ls or Glob to confirm every path exists before listing it.
   f. Commands: copy the scripts exactly from package.json with one line
      each.
   g. Data: challenge fixture goes to data/fixture.json (absent unless it
      exists now - check), canonical input format documented in
      docs/fixture-notes.md, demo data is synthetic and never scored,
```

```
      demo option is shown in dev or when VITE_DEMO_MODE=true.
   h. Contract rules for src/engine/types.ts and tests/contract.test.ts.
   i. Code rules: small files, no dead code, no console.log, tests for every
      changed business rule, npm test + typecheck + build before commits.
   j. Commit message style (lowercase, 3-8 words, no symbols, no AI
      mention, no co-author trailers).
   k. Honesty rules: never invent results, URLs, scores or labels.
3. Keep it under about 160 lines. The content must not mention AI tools.
4. Do not edit C:\Users\shiva\.claude\CLAUDE.md (my user-level file).

FILES AND MODULES
Create: CLAUDE.md. Read: README.md, package.json, docs/integration-status.md, docs/fixture-notes.md,
C:\Users\shiva\projects\followthru\CLAUDE.md.

INTEGRATION REQUIREMENTS
Claude Code loads CLAUDE.md automatically in every new session, so later prompts and the CONTINUE prompt rely on
it. It must match the code exactly.

TESTING REQUIREMENTS
Documentation only. Still run npm test, npm run typecheck and
npm run build to confirm the tree is healthy, and check every path you
listed exists (for example with a small loop over the listed paths).

COMMITS (push to origin full-integration after each)
add project memory for single laptop

ACCEPTANCE CRITERIA
- CLAUDE.md exists in the repository root and is pushed.
- Every file path it lists exists; every command matches package.json.
- No two-laptop instructions remain; no mention of AI tools in its text.
- npm test, typecheck and build still pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open CLAUDE.md on GitHub (branch full-integration) and read it once.
- Confirm it says work happens on full-integration and main needs the exact phrase.
- Start a new Claude Code chat and ask: what branch do we work on? It should answer from CLAUDE.md.

Next prompt dependency. None. C01 should run first because every later prompt reads CLAUDE.md.

C02 Challenge fixture intake, fixture notes and adapter

Blocked until you provide the fixture Continues: P03, adapter part of P04

Goal. Load the real Fynd fixture, document every field and edge case with message ids, and make it pass the existing validator (through a small adapter only if its field names differ).

Current context. data/ contains only README.md. docs/fixture-notes.md is a template with a canonical input format section. Validation lives in src/schema/validate.ts (validateDataset, validateDatasetJson); the Setup page bundles data/fixture.json through import.meta.glob when the file exists; scripts/evidence.ts reads data/fixture.json by default; api/_lib/handlers.ts validates POST bodies with the same validator.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C02 - Challenge fixture intake, fixture notes and adapter

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
```

```
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Load the real Fynd fixture, document every field and edge case with message ids, and make it pass the existing
validator (through a small adapter only if its field names differ).

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
data/ contains only README.md. docs/fixture-notes.md is a template with a canonical input format section.
Validation lives in src/schema/validate.ts (validateDataset, validateDatasetJson); the Setup page bundles
data/fixture.json through import.meta.glob when the file exists; scripts/evidence.ts reads data/fixture.json by
default; api/_lib/handlers.ts validates POST bodies with the same validator.

BEFORE THIS PROMPT I copied Fynd's file Starter_Data/C14/fixture.json to
D:\followthru\data\fixture.json.

STEP 0. If data/fixture.json does not exist, STOP. Reply with one line:
"BLOCKED: data/fixture.json missing" and change nothing.

TASK: understand the real starter data and make the app accept it. Do NOT
change engine rules in this prompt (C03 does that).

1. Never modify data/fixture.json. Ask me whether it may be committed
   (it is needed for a deployed build to offer "Load the C14 challenge
   fixture"). Until I answer yes, do not stage it.
2. Read the whole file (every record, not a sample) and rewrite
   docs/fixture-notes.md. Keep the "Canonical input format" section and
   replace the template sections with:
   a. top-level structure with counts;
   b. a table of every field: path, type, required, example, meaning;
   c. people with ids, names, timezones, roles or teams;
   d. threads with ids, titles, participants and message counts;
   e. how timestamps and timezones are written, and any message whose
      local date differs from its UTC date;
   f. whether commitments, due dates, changes and completions are
      structured fields, free text, or both;
   g. where the labelled expected answers live, their exact format and
      status vocabulary, and a table listing every labelled item;
   h. the evaluation clock (explicit reference time, or a proposal such as
      the latest message time, with the reason);
   i. an edge-case catalogue with every message id and thread id and a short
      quote for: tentative deadline, timezone difference, delegation,
      completion in another thread, deadline changed later, cancelled,
      deferred or on hold, silence after deadline, explicit not done yet,
      anything ambiguous or messy;
   j. data quality problems.
3. Validate it: write a tiny tsx one-off (not committed) that runs
   validateDatasetJson on the file and prints the issues.
   - If it validates with zero errors, no adapter is needed.
   - If field names or shapes differ, create src/schema/adapter.ts with
```

```
     a pure function adaptFixture(raw: unknown): unknown that maps the real
     shape onto the canonical DatasetInput (people, threads, messages,
     labels, referenceTime). Return the input unchanged when it already
     validates. Map the label status vocabulary to open, done, cancelled or
     deferred with a documented table; an unknown status must produce a
     validation error, never a guess. Never change label values beyond that
     vocabulary mapping. Document the field mapping in fixture-notes.
   - Wire the adapter in exactly three places: the fixture loader in
     src/pages/Setup.tsx (before validateDatasetJson), scripts/evidence.ts
     (before validation) and api/_lib/handlers.ts handleProcess (before
     validateDataset, accept body key "fixture" as an alias of "dataset").
     Export it from src/schema/index.ts.
4. Tests: tests/adapter.test.ts with a small excerpt that uses the REAL
   field names copied from the fixture (two or three records, not invented
   fields) and checks the mapping; plus a test that reads data/fixture.json
   and expects zero validation errors, using it.skipIf when the file is
   absent so the suite still passes without it.
5. Run npm run evidence and paste the full scorecard summary it prints.
   Do not fix engine failures here; list them for C03.

FILES AND MODULES
Read: data/fixture.json, src/schema/validate.ts, src/schema/input.ts, src/pages/Setup.tsx, scripts/evidence.ts,
api/_lib/handlers.ts. Modify: docs/fixture-notes.md, src/schema/index.ts. Create if needed:
src/schema/adapter.ts, tests/adapter.test.ts.

INTEGRATION REQUIREMENTS
The adapter is the only place that knows the real fixture shape. Everything after it keeps using the canonical
Dataset type from src/engine/types.ts, so the engine, store, UI and API stay unchanged.

TESTING REQUIREMENTS
npm test (including the new adapter tests), npm run typecheck,
npm run build, npm run evidence. In the browser: npm run dev, open
/setup and confirm the "Load the C14 challenge fixture" button appears and
validates with zero errors.

COMMITS (push to origin full-integration after each)
add real fixture notes
add fixture adapter (only if needed)
validate challenge fixture in tests

ACCEPTANCE CRITERIA
- data/fixture.json validates with zero errors (directly or via the adapter).
- docs/fixture-notes.md lists every labelled commitment with message ids
  and the full edge-case catalogue.
- npm run evidence produced a scorecard; its summary is in your report.
- No engine file under src/engine changed.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open docs/fixture-notes.md: the labelled items and edge cases must cite real message ids.
- Run npm run dev, open http://localhost:5173/setup and load the challenge fixture.
- Read the scorecard summary in Claude's report. Answer its question about committing the fixture.

Next prompt dependency. Needs C01. Needs the real fixture file from you. If you do not have it yet, skip to C04 and return here as soon as you get it.

C03 Make the engine pass the labelled commitments

Blocked until C02 Continues: Real-data verification of P06, P07, P08, P09

Goal. Reach a correct owner, deadline and status for every labelled commitment by fixing general rules (never hardcoded answers), with a regression test for each fix.

Current context. The engine has only been verified on synthetic data (tests/engine.test.ts, src/mocks/demoDataset.json). Known design choices to check against the real labels: a request addressed to one person opens a commitment even before they reply (response pending); date-only deadlines resolve to 23:59 in the speaker's timezone and EOD to 18:00; next Friday means the first Friday strictly after today; cross-thread links use keyword overlap thresholds STRONG_LINK 0.5 and WEAK_LINK 0.3 in src/engine/history.ts; evaluate.ts compares deadlines within 60 seconds.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C03 - Make the engine pass the labelled commitments

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
```

```
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Reach a correct owner, deadline and status for every labelled commitment by fixing general rules (never hardcoded
answers), with a regression test for each fix.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
The engine has only been verified on synthetic data (tests/engine.test.ts, src/mocks/demoDataset.json). Known
design choices to check against the real labels: a request addressed to one person opens a commitment even before
they reply (response pending); date-only deadlines resolve to 23:59 in the speaker's timezone and EOD to 18:00;
next Friday means the first Friday strictly after today; cross-thread links use keyword overlap thresholds
STRONG_LINK 0.5 and WEAK_LINK 0.3 in src/engine/history.ts; evaluate.ts compares deadlines within 60 seconds.

STEP 0. If data/fixture.json does not exist, STOP with
"BLOCKED: data/fixture.json missing".

TASK: make every labelled commitment correct by improving general rules.

1. Run npm run evidence and open the newest scorecard.md in evidence/.
2. For every failing labelled item, inspect the commitment's events
   (rule names and evidence message ids) and classify the cause:
   signal detection (src/engine/signals.ts), addressee resolution or
   linking (src/engine/history.ts), deadline parsing (src/engine/
   deadline.ts), title keywords used for linking (titleFrom, keywords),
   authority rules, responseState, the evaluation tolerance, or the label
   adapter mapping (src/schema/adapter.ts).
3. Print a diagnosis table BEFORE changing code: label id, expected,
   actual, cause, proposed general fix, risk to other cases.
4. Fix one cause at a time:
   - Never special-case message ids, people ids or label values in
     src/engine. After all fixes, grep src/engine for every labelled
     message id; there must be zero matches.
   - Add a regression test per fix. If the fixture is committed, tests may
     read the real messages; otherwise build a minimal synthetic excerpt
     that reproduces the same wording pattern and label it synthetic.
   - After each fix run npm test and npm run evidence. Existing synthetic
     expectations in tests/engine.test.ts must keep passing; if one must
     change, explain exactly why in the commit's report.
5. Stop when all checks pass, or when a remaining failure needs a human
   decision (for example a label that contradicts the brief's rules).
   Report those with the evidence and do not force them.
6. Keep the final passing run folder from evidence/ and commit it as
   challenge evidence (generated by the script, never edited by hand).

FILES AND MODULES
Modify as needed: src/engine/signals.ts, src/engine/history.ts, src/engine/deadline.ts, src/engine/drafts.ts,
```

```
src/schema/adapter.ts, tests/engine.test.ts or new tests/fixture-cases.test.ts. Read: evidence/*/scorecard.md,
docs/fixture-notes.md.

INTEGRATION REQUIREMENTS
All fixes stay inside the pure engine, so the dashboard, queue, API and evidence script pick them up
automatically. If a fix needs a new field, add it to src/engine/types.ts additively and update
tests/contract.test.ts only if exports change.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run evidence after every fix; npm run e2e once at the end.

COMMITS (push to origin full-integration after each)
one per fix, for example: link completion across threads by owner
read deadline phrase in owner timezone
save passing challenge evidence

ACCEPTANCE CRITERIA
- Scorecard: all labelled commitments correct (or every remaining failure
  documented with evidence and agreed with me), 0 active reminders on
  closed work, no duplicate drafts.
- grep of labelled message ids in src/engine returns nothing.
- A regression test exists for every fix; all suites pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open the committed evidence folder's scorecard.md and compare with the labels in docs/fixture-notes.md.
- Open the dashboard with the fixture loaded and spot-check two commitments in the detail page's Why? timeline.

Next prompt dependency. Needs C02. Re-run this prompt after C04 and C05 if the fixture arrives before them, because parser and signal changes can shift results.

C04 Deadline parser: explicit timezones, relative and numeric dates

Essential Continues: Remaining part of P05

Goal. Cover the deadline forms the original plan required but the parser does not handle yet, without changing any verified behaviour.

Current context. src/engine/deadline.ts parseDeadline(text, { saidAt, tz }) supports weekdays with this/next/coming, today, tonight, tomorrow, day after tomorrow, EOD/COB/end of day (18:00), end of week (Friday 18:00), ISO dates, month-name dates (Oct 20, 20th October), times (6pm, 18:00, noon, morning, afternoon, evening), and bare hours with -ish. Date-only deadlines get 23:59 local (DATE_ONLY_TIME). Hedges live in HEDGE_RE and make certainty tentative. src/engine/time.ts wallTimeToUtcMs handles DST (gap moves forward, overlap takes the later instant). tests/deadline.test.ts and tests/time.test.ts hold 33 passing cases. The deadline rules table is in docs/track-a-status.md.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C04 - Deadline parser: explicit timezones, relative and numeric dates

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
```

```
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Cover the deadline forms the original plan required but the parser does not handle yet, without changing any
verified behaviour.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/engine/deadline.ts parseDeadline(text, { saidAt, tz }) supports weekdays with this/next/coming, today,
tonight, tomorrow, day after tomorrow, EOD/COB/end of day (18:00), end of week (Friday 18:00), ISO dates, month-
name dates (Oct 20, 20th October), times (6pm, 18:00, noon, morning, afternoon, evening), and bare hours with -
ish. Date-only deadlines get 23:59 local (DATE_ONLY_TIME). Hedges live in HEDGE_RE and make certainty tentative.
src/engine/time.ts wallTimeToUtcMs handles DST (gap moves forward, overlap takes the later instant).
tests/deadline.test.ts and tests/time.test.ts hold 33 passing cases. The deadline rules table is in docs/track-a-
status.md.

TASK: extend parseDeadline. Keep every existing test passing unchanged.

1. Explicit timezone in the phrase overrides the speaker's timezone:
   "5pm IST", "9am PT", "14:00 UTC", "10am GMT", "6pm BST", "9am ET",
   "4pm CET", "11am SGT". Add an abbreviation map to IANA names in
   src/engine/time.ts: IST -> Asia/Kolkata, PT/PST/PDT ->
   America/Los_Angeles, ET/EST/EDT -> America/New_York, UTC and GMT -> UTC,
   BST -> Europe/London, CET/CEST -> Europe/Paris, SGT -> Asia/Singapore.
   Document that IST is read as India. The returned interpretedTz must be
   the zone actually used. The calendar day of a weekday phrase is still
   resolved from the speaker's "today", then the time is applied in the
   explicit zone (document this in the code comment and the table).
2. Relative durations: "in 2 days", "in 3 hours", "in a week", "within
   48 hours". Days keep the date-only end-of-day rule; hours are exact.
3. "next week" -> Monday of next week, date-only rule (23:59 local).
   "by end of next week" -> Friday of next week, 18:00.
4. Numeric dates: "12/10", "12/10/2026", "12-10-2026" read as day/month
   (India and UK convention). If the value is invalid as day/month but
   valid as month/day, use month/day. Year inference uses the existing
   closestYear helper.
5. More hedges that make certainty tentative: "i think", "around",
   "roughly", "approx", "approximately", and a "?" that directly follows
   the date or time phrase ("Friday?"). Make sure "at the latest" and
   "by Friday sharp" stay firm (negative tests).
6. Update the deadline rules table in docs/track-a-status.md with every
   new rule and the documented assumptions.
7. Tests in tests/deadline.test.ts, at least 25 new cases, including:
   - "by Monday 10am PT" said by an IST speaker uses Pacific time;
   - a message sent late Thursday UTC that is already Friday in IST;
   - "tomorrow EOD" across a month end;
```

```
   - "in 2 days" across a DST change in America/Los_Angeles;
   - "12/10" versus "13/10" versus "10/13";
   - each new hedge, plus the firm negatives;
   - all old cases still pass unchanged.
8. Run the full suite. tests/engine.test.ts must keep passing without
   edits; if any demo expectation changes, stop and explain why.

FILES AND MODULES
Modify: src/engine/deadline.ts, src/engine/time.ts, tests/deadline.test.ts, tests/time.test.ts, docs/track-a-
status.md. Export new helpers from src/engine/index.ts only if other modules need them (then update
tests/contract.test.ts).

INTEGRATION REQUIREMENTS
parseDeadline is used by src/engine/history.ts for requests, promises, acceptances and reschedules, so every
screen and the API pick up the new forms automatically. No UI change is needed.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build. Then npm run evidence -- --demo and confirm the synthetic scorecard
still shows 3 drafts and 0 active reminders on closed work.

COMMITS (push to origin full-integration after each)
read explicit timezones in deadline phrases
parse relative and numeric dates
widen tentative deadline hedges

ACCEPTANCE CRITERIA
- 25 or more new deadline tests pass; all previous tests pass unchanged.
- interpretedTz reflects an explicit zone when one is given.
- docs/track-a-status.md table lists every new rule and assumption.
- npm run evidence -- --demo output unchanged (3 drafts, check 2 and 3 PASS).

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Run npm test and look for the new deadline cases in the output.
- Run npm run evidence -- --demo: still 3 drafts and PASS for the reminder and duplicate checks.

Next prompt dependency. Needs C01. Independent of the fixture. If C03 already ran, re-run C03 afterwards.

C05 Message signals: progress, not done, blockers, reopen, extractor seam

Essential Continues: Remaining parts of P06 and P07

Goal. Record owner replies such as 'still working on it' as evidence without changing status, reopen work when a completion is retracted, and prepare a clean seam for the optional AI extractor.

Current context. src/engine/signals.ts detectSignals returns request, commit, accept, done, cancel, defer, reschedule, delegate and decline (DONE_NEGATED stops 'not sent yet' from counting). src/engine/history.ts applyUpdate applies them with authority checks. An owner message such as 'still working on it' only updates the internal lastOwnerMsgAt (so responseState becomes responded) and adds no event. There is no reopen path for 'actually it is not done'. CommitmentEventType in src/engine/types.ts has no progress, not_done or blocker types. Blockers currently come only from human decisions (report_blocker). The detail page labels events through EVENT_LABEL in src/pages/CommitmentDetail.tsx.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C05 - Message signals: progress, not done, blockers, reopen, extractor seam

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
```

```
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Record owner replies such as 'still working on it' as evidence without changing status, reopen work when a
completion is retracted, and prepare a clean seam for the optional AI extractor.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/engine/signals.ts detectSignals returns request, commit, accept, done, cancel, defer, reschedule, delegate
and decline (DONE_NEGATED stops 'not sent yet' from counting). src/engine/history.ts applyUpdate applies them
with authority checks. An owner message such as 'still working on it' only updates the internal lastOwnerMsgAt
(so responseState becomes responded) and adds no event. There is no reopen path for 'actually it is not done'.
CommitmentEventType in src/engine/types.ts has no progress, not_done or blocker types. Blockers currently come
only from human decisions (report_blocker). The detail page labels events through EVENT_LABEL in
src/pages/CommitmentDetail.tsx.

TASK: three small, tested engine additions. No status may change from
silence or from a progress note.

1. Types (additive) in src/engine/types.ts: add 'progress', 'not_done',
   'blocker' to CommitmentEventType.
2. src/engine/signals.ts: add signal kinds
   - not_done: "not yet", "still working", "running late", "need more
     time", "not done", "not finished";
   - blocker: "blocked by", "waiting on", "can't until", "stuck on",
     "waiting for";
   - progress: "making progress", "half way", "halfway", "almost there",
     "first draft ready", "nearly done";
   - reopen: "not done after all", "still broken", "reopening", "that was
     the wrong file", "please redo", "actually it's not done".
   Keep the existing DONE_NEGATED behaviour; "almost done" must be
   progress, never done.
3. src/engine/history.ts applyUpdate:
   - owner (or accepted delegate) messages with not_done, progress or
     blocker add the matching event with evidence; status unchanged;
   - blocker messages also append a Blocker { at, by, text: short quote,
     resolved: false } to commitment.blockers;
   - reopen after done: by the requester, a coordinator or the owner ->
     status open, event 'reopened' with evidence; by anyone else ->
     needsReview with reason 'conflicting_evidence';
   - the existing draft logic in src/engine/drafts.ts already reactivates
     a suppressed draft as a new version when work is open and overdue;
     verify it, do not duplicate it.
4. Extractor seam: create src/engine/extractor.ts with
```

```
   export interface Extractor { name: 'rules' | 'ai'; detect(text: string):
   Signal[] } and export const rulesExtractor. resolveCommitments and
   runEngine accept an optional extractor argument that defaults to
   rulesExtractor. No behaviour change for existing callers.
5. UI: add labels for the new event types in EVENT_LABEL in
   src/pages/CommitmentDetail.tsx (Progress update, Not done yet,
   Blocker reported).
6. Tests in tests/engine.test.ts using the existing withMessages helper and
   clearly synthetic messages:
   - "still working on it, need two more days" after the deadline: status
     open, responseState responded, a not_done event, still drafted;
   - a blocker message adds a blocker and an event;
   - "almost done" is not completion;
   - reopen after done by the requester reactivates the SAME draft id with
     a higher version and cause 'reactivated';
   - reopen claim by an unrelated person goes to review;
   - an injected fake extractor is used when passed.
7. Update tests/contract.test.ts only if index.ts exports change, and log
   the type additions in docs/integration-status.md.

FILES AND MODULES
Modify: src/engine/types.ts, src/engine/signals.ts, src/engine/history.ts, src/engine/run.ts,
src/engine/index.ts, src/pages/CommitmentDetail.tsx, tests/engine.test.ts, tests/contract.test.ts,
docs/integration-status.md. Create: src/engine/extractor.ts.

INTEGRATION REQUIREMENTS
New events appear automatically in the detail page timeline and in exports (commitments CSV evidence ids).
Blockers from messages appear in the Team page blockers list, which already reads commitment.blockers.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run evidence -- --demo (still 3 drafts unless you can explain a
change), npm run e2e.

COMMITS (push to origin full-integration after each)
detect progress and blocker messages
reopen work when completion is retracted
add extractor interface for rules

ACCEPTANCE CRITERIA
- New signals detected and tested, including the negative "almost done".
- Progress or not-done replies never change status; reopen works with
  the authority rules; one draft per commitment still holds.
- runEngine works with and without an explicit extractor.
- All suites pass; contract changes logged.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Run npm run dev, load the demo, open a commitment's detail page and confirm the timeline still renders.
- Read the new test names in the npm test output.

Next prompt dependency. Needs C01. Re-run C03 afterwards if the fixture is available.

C06 Reminder wording by response state, subjects and escalation recipient

Essential Continues: Remaining parts of P08 and P11

Goal. Make drafts say different things to someone who never replied and someone who replied 'not yet', add short subjects, and let a flow step escalate to the requester or coordinator while keeping one draft per commitment.

Current context. src/engine/drafts.ts composeBody(c, kind, tone, people, now) ignores responseState; there is no subject. The escalate tone mentions the requester in the text but recipientId is always the owner. src/engine/approval.ts checkBeforeApproval treats draft.recipientId !== commitment.ownerId as stale, so a different recipient would be blocked. FlowStep is { afterHours, tone }. DEFAULT_FLOW in src/engine/flows.ts has steps 0h gentle, 24h firm, 72h escalate. The Flows page (src/pages/Flows.tsx) edits steps; the Queue page (src/pages/Queue.tsx) shows draft body and recipient.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C06 - Reminder wording by response state, subjects and escalation recipient

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
```

```
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Make drafts say different things to someone who never replied and someone who replied 'not yet', add short
subjects, and let a flow step escalate to the requester or coordinator while keeping one draft per commitment.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/engine/drafts.ts composeBody(c, kind, tone, people, now) ignores responseState; there is no subject. The
escalate tone mentions the requester in the text but recipientId is always the owner. src/engine/approval.ts
checkBeforeApproval treats draft.recipientId !== commitment.ownerId as stale, so a different recipient would be
blocked. FlowStep is { afterHours, tone }. DEFAULT_FLOW in src/engine/flows.ts has steps 0h gentle, 24h firm, 72h
escalate. The Flows page (src/pages/Flows.tsx) edits steps; the Queue page (src/pages/Queue.tsx) shows draft body
and recipient.

TASK: better draft wording and an optional escalation recipient.

1. Types (additive): Draft.subject?: string; DraftVersion.subject?: string;
   FlowStep.recipient?: 'owner' | 'requester' | 'coordinator' (missing
   means owner).
2. src/engine/drafts.ts:
   - composeSubject(...) under 60 characters, for example
     "Quick check: <task>" or "Date needed: <task>".
   - composeBody variants, each under 70 words, polite, specific, using
     the recipient's first name and the deadline in the recipient's
     timezone:
     a. reminder + responseState pending: "haven't heard back";
     b. reminder + the latest event is not_done or progress (from C05):
        ask for a realistic new date;
     c. clarify: confirm a firm date for the tentative phrase;
     d. escalation to requester or coordinator: factual summary naming
        the owner, the deadline, how long overdue and the last reply time.
   - export function expectedRecipient(c, step, people): owner by default;
     'requester' -> c.requesterId, falling back to the coordinator;
     'coordinator' -> the first person with role coordinator sorted by id.
   - buildDrafts uses expectedRecipient. A recipient change between steps
     already bumps the version with cause 'recipient_changed' and resets
     recipientConfirmed - verify it with a test.
3. src/engine/approval.ts: replace the ownerId comparison with
   expectedRecipient so escalation drafts are not wrongly stale, while a
   changed owner on an owner-addressed draft is still stale.
4. Keep DEFAULT_FLOW unchanged (all steps to the owner) so verified
   behaviour does not move. Add the recipient choice to the step editor in
   src/pages/Flows.tsx and to validateFlow (escalating to the requester
   when there is none is allowed but warned in the preview).
```

```
5. Optional subtask if time allows: quiet hours. Flow gains optional
   quietHours { start: 'HH:mm', end: 'HH:mm' } in the recipient's
   timezone; a step that becomes due inside quiet hours waits until the end
   of quiet hours. Pure, tested, editable in Flows. Skip it if any core
   test becomes hard to keep green and say so.
6. Queue card shows the subject above the body.
7. Tests: wording differs for pending versus not_done; subject length;
   a custom flow escalating to the coordinator at step 2 produces the same
   draft id, version + 1, recipient = coordinator, recipientConfirmed
   false, and approval passes after confirmation; owner-change staleness
   still detected; demo expectations unchanged.

FILES AND MODULES
Modify: src/engine/types.ts, src/engine/drafts.ts, src/engine/approval.ts, src/engine/flows.ts,
src/engine/index.ts, src/pages/Flows.tsx, src/pages/Queue.tsx, tests/engine.test.ts, tests/contract.test.ts,
docs/integration-status.md.

INTEGRATION REQUIREMENTS
Queue cards, CSV exports (draftsToCsv) and the API all read Draft, so subjects appear everywhere after this
change. Recipient review warnings (recipientWarnings) keep working for the new recipients.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e (the queue approval journey must still pass).

COMMITS (push to origin full-integration after each)
word reminders by reply state
add draft subjects
let flow steps escalate to coordinator

ACCEPTANCE CRITERIA
- Different, tested wording for pending, not done, clarify and escalation.
- Escalation drafts keep one draft per commitment and can be approved after
  confirmation; owner-change staleness still blocks.
- DEFAULT_FLOW behaviour and demo results unchanged; all suites pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- In the app, open Flows, create a flow for everyone with step 2 escalating to the coordinator, then open Queue and check the recipient and subject.
- Delete the test flow afterwards (Flows page) so the demo returns to normal.

Next prompt dependency. Needs C05 (uses its not_done and progress events).

C07 Product change record: starting state, scope and final evidence

Essential Continues: Remaining parts of P09 and P10

Goal. Produce the three items the brief requires for the negotiated change (starting state, negotiated scope, final evidence), backed by real script runs with the flags off and on.

Current context. scripts/evidence.ts supports --input, --demo and --now and writes evidence/<demo|run>-<stamp>/ with scorecard.md, results.json, commitments.csv, drafts.csv, validation.json and manifest.json (SHA-256 per file). It has no --label, no flag switch, no evidence/latest copy, no test report, no git commit hash and no suppressions CSV. .gitignore ignores evidence/demo-*/. In the engine, EngineFlags { staleSuppression, recipientReview, ai } exist, but buildDrafts always suppresses drafts of closed work and checkBeforeApproval always re-checks freshness; only time-based staleness and the recipient confirmation honour the flags. No docs/product-change.md exists.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C07 - Product change record: starting state, scope and final evidence

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
```

```
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Produce the three items the brief requires for the negotiated change (starting state, negotiated scope, final
evidence), backed by real script runs with the flags off and on.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
scripts/evidence.ts supports --input, --demo and --now and writes evidence/<demo|run>-<stamp>/ with scorecard.md,
results.json, commitments.csv, drafts.csv, validation.json and manifest.json (SHA-256 per file). It has no --
label, no flag switch, no evidence/latest copy, no test report, no git commit hash and no suppressions CSV.
.gitignore ignores evidence/demo-*/. In the engine, EngineFlags { staleSuppression, recipientReview, ai } exist,
but buildDrafts always suppresses drafts of closed work and checkBeforeApproval always re-checks freshness; only
time-based staleness and the recipient confirmation honour the flags. No docs/product-change.md exists.

TASK: make the flags produce a real before/after comparison and save the
evidence the brief asks for.

1. Flag semantics (engine, defaults stay ON everywhere in the app):
   - staleSuppression = false (baseline): buildDrafts keeps a previously
     created draft unchanged when its situation later changes (no
     auto-suppression of an existing draft, no time-based stale marking);
     drafts are still never CREATED for closed work. checkBeforeApproval
     skips the freshness re-check (terminal status, changed deadline or
     owner, age) and only checks the draft state.
   - recipientReview = false: approval does not need recipientConfirmed.
   - Tests in tests/engine.test.ts: with staleSuppression false, a draft
     created before a completion message stays pending_review after the
     completion (the baseline problem); with true it is suppressed with
     reason completed and evidence. The default config result must be
     byte-identical to before this prompt for the demo data.
2. scripts/evidence.ts:
   - --label <name>: output to evidence/runs/<YYYY-MM-DD_HH-mm-ss>_<label>/
     for fixture runs; synthetic runs go to evidence/demo-runs/<same>/ and
     stay ignored by git unless copied into starting_state or final.
   - --flags on|off (default on).
   - --simulate-later <ISO cutoff>: first run with messages at or before
     the cutoff and now = cutoff, then a second run with all messages and
     the first run's drafts as previousDrafts; write stale-report.md listing
     every draft that became outdated (and, with flags off, which would
     still be approved and sent wrongly).
   - test-report.txt: output of npx vitest run --reporter=verbose; record
     a failure honestly and exit non-zero.
   - suppressions.csv via a new suppressionsToCsv(drafts) in
     src/schema/io.ts (draft id, commitment id, reason, at, evidence message
     id), formula-escaped like the other exports, with a unit test in
```

```
     tests/io.test.ts.
   - manifest.json adds label, flags, engine mode, dataset name, whether
     the data is synthetic, git commit hash and whether the tree was dirty
     (git rev-parse --short HEAD, git status --porcelain).
   - copy the run to evidence/latest/ (replace its contents).
3. Runs (use data/fixture.json if it exists, otherwise --demo and label
   everything SYNTHETIC in titles and READMEs):
   - starting state: --label starting-state --flags off --simulate-later
     with a cutoff before the completion messages (for the demo, use
     2026-10-16T08:00:00Z, before the cross-thread completion m_10);
     copy to evidence/starting_state/ and write its README.md: how many
     drafts, for which commitments, which became outdated, that recipients
     are not reviewed.
   - final: same command with --flags on, label final; copy to
     evidence/final/ and write README.md comparing before and after.
4. docs/product-change.md: starting state (link), the problem (stale drafts,
   wrong recipients), negotiated scope IN (pre-send freshness check,
   automatic suppression or update with visible reason, staleness after
   staleAfterHours, recipient review with warnings and confirmation, both
   toggles for comparison), scope OUT (real sending, Slack or email
   integration, ML ranking, multi-recipient drafts), numbered acceptance
   criteria each mapped to a test name, and a Final evidence section.
5. Commit evidence/starting_state/, evidence/final/ and evidence/latest/
   even when synthetic, because their READMEs say so. When the real fixture
   arrives later, re-run this prompt's step 3 and replace them.

FILES AND MODULES
Modify: src/engine/drafts.ts, src/engine/approval.ts, scripts/evidence.ts, src/schema/io.ts, src/schema/index.ts,
tests/engine.test.ts, tests/io.test.ts, .gitignore. Create: docs/product-change.md,
evidence/starting_state/README.md, evidence/final/README.md.

INTEGRATION REQUIREMENTS
The Evidence page settings already toggle staleSuppression and recipientReview, so the comparison is also visible
in the app. The default config stays unchanged for all screens and the API.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, the two evidence commands from step 3, npm run e2e.

COMMITS (push to origin full-integration after each)
make stale suppression follow its flag
add labelled evidence runs with test report
record product change scope and evidence

ACCEPTANCE CRITERIA
- Flags off reproduces the stale-draft problem; flags on fixes it; both
  are covered by tests; default results unchanged.
- evidence/starting_state, evidence/final and evidence/latest exist with
  manifest hashes, a test report and clearly stated data source.
- docs/product-change.md has starting state, scope IN and OUT, acceptance
  criteria mapped to tests, and final evidence links.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open docs/product-change.md and both READMEs; confirm they say SYNTHETIC if the fixture was not used.
- Open evidence/final/stale-report.md and compare it with evidence/starting_state/stale-report.md.

Next prompt dependency. Needs C06. Re-run step 3 after C03 when the fixture is available.

C08 Draft review actions: reject, edit and clarification lane

Essential Continues: Remaining part of P20 (queue)

Goal. Let a coordinator reject or edit a draft (both audited and preserved across engine runs) and see tentative-deadline drafts in their own lane.

Current context. DraftState is pending_review | approved | suppressed | stale. src/store/actions.ts has approve(draftId) (runs checkBeforeApproval, suppresses when stale) and confirmRecipient. src/pages/Queue.tsx has tabs Needs review, Approved, Stale, Suppressed; clarify drafts sit in Needs review. Approved state survives engine runs because buildDrafts keeps an unchanged previous draft. src/components/domain/view.ts draftStateView maps states to chips; src/engine/evaluate.ts treats pending_review and approved as active.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C08 - Draft review actions: reject, edit and clarification lane

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
```

```
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Let a coordinator reject or edit a draft (both audited and preserved across engine runs) and see tentative-
deadline drafts in their own lane.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
DraftState is pending_review | approved | suppressed | stale. src/store/actions.ts has approve(draftId) (runs
checkBeforeApproval, suppresses when stale) and confirmRecipient. src/pages/Queue.tsx has tabs Needs review,
Approved, Stale, Suppressed; clarify drafts sit in Needs review. Approved state survives engine runs because
buildDrafts keeps an unchanged previous draft. src/components/domain/view.ts draftStateView maps states to chips;
src/engine/evaluate.ts treats pending_review and approved as active.

TASK: reject and edit with full audit, plus a clarification lane.

1. Types (additive): DraftState adds 'rejected'. Draft adds optional
   rejectedBy, rejectedAt, rejectReason, editedBody, editedBy, editedAt.
   Fix every exhaustive mapping that TypeScript flags (view.ts, Queue,
   evaluate ACTIVE list keeps rejected inactive).
2. Pure functions in src/engine/approval.ts: rejectDraft(draft, by, at,
   reason) and editDraft(draft, by, at, body). Body 1 to 600 characters,
   trimmed.
3. src/engine/drafts.ts buildDrafts: a rejected or edited draft stays as it
   is while nothing changes; when a cause creates a new version, the draft
   returns to pending_review, the edit is dropped and the history entry
   says "edit replaced by new version (<cause>)". Approval and exports use
   editedBody when present.
4. src/store/actions.ts: reject(draftId, reason) and edit(draftId, body),
   coordinator only (canDo 'approve_draft'), reason required for reject,
   audit through repo.putDraft with actions draft.reject and draft.edit.
5. src/pages/Queue.tsx:
   - Reject button opens a Dialog with a required reason.
   - Edit switches the body to a Textarea with Save and Cancel; edited
     drafts show an "Edited" note.
   - Lanes: Needs review (reminders pending), Needs clarification (clarify
     pending), Approved (shadow), Stale, Suppressed, Rejected.
   - Header line computed from the scorecard: "<n> active reminders for
     completed, cancelled or deferred work" (should be 0).
   - Optional: keyboard shortcuts J/K next/previous card, A approve,
     R reject, E edit, only when no input has focus.
6. Tests: engine (rejected survives re-run; new version returns to review;
   edit preserved then replaced), store (member cannot reject or edit;
   reason required), UI test in tests/ui.test.tsx (reject moves the card
   to Rejected), and extend e2e/app.spec.ts with an edit then reject step.

FILES AND MODULES
Modify: src/engine/types.ts, src/engine/approval.ts, src/engine/drafts.ts, src/engine/evaluate.ts,
```

```
src/engine/index.ts, src/store/actions.ts, src/pages/Queue.tsx, src/components/domain/view.ts,
tests/engine.test.ts, tests/store.test.ts, tests/ui.test.tsx, e2e/app.spec.ts, tests/contract.test.ts,
docs/integration-status.md.

INTEGRATION REQUIREMENTS
Draft persistence is handled by syncDrafts in src/store/repo.ts, which writes whatever the engine returns, so
human states must be preserved by buildDrafts itself. The audit log shows each action on the Evidence page.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e.

COMMITS (push to origin full-integration after each)
add reject and edit for drafts
show clarification lane in queue

ACCEPTANCE CRITERIA
- Reject needs a reason, edit is validated, both audited and coordinator-only.
- Both survive engine re-runs until a new version is created.
- Six lanes visible; header count shows 0 for the demo; all suites pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- As Priya, edit a draft, reload the page and confirm the edit stayed.
- Reject another draft with a reason; check it in the Rejected lane and in Evidence > Audit log.

Next prompt dependency. Needs C06 (subjects shown on cards) and C07 (flag-aware approval).

C09 Role-based navigation and member workspace gaps

Essential Continues: Remaining parts of P12, P17 and P21

Goal. Show each role only the screens it can use, let members ask for more time (never auto-applied), and give coordinators a people grid and an extension inbox.

Current context. src/components/shell/AppShell.tsx guards for data and profile. Sidebar.tsx, BottomTabBar.tsx and MobileNavSheet.tsx show all routes to everyone. Queue filters drafts to the member's own (read only). Team.tsx has a coordinator view (assign form, submissions inbox, blockers) and a member view (own items). Member actions (progress, blocker, submit) live in CommitmentDetail.tsx. Permissions: canDecide and canDo in src/engine/permissions.ts. Decisions are applied in applyDecision in src/engine/history.ts.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C09 - Role-based navigation and member workspace gaps

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
```

```
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Show each role only the screens it can use, let members ask for more time (never auto-applied), and give
coordinators a people grid and an extension inbox.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/components/shell/AppShell.tsx guards for data and profile. Sidebar.tsx, BottomTabBar.tsx and
MobileNavSheet.tsx show all routes to everyone. Queue filters drafts to the member's own (read only). Team.tsx
has a coordinator view (assign form, submissions inbox, blockers) and a member view (own items). Member actions
(progress, blocker, submit) live in CommitmentDetail.tsx. Permissions: canDecide and canDo in
src/engine/permissions.ts. Decisions are applied in applyDecision in src/engine/history.ts.

TASK: role-aware navigation plus the missing member and coordinator
pieces.

1. Navigation by role (one shared list used by Sidebar, BottomTabBar and
   MobileNavSheet, defined once in src/components/shell/nav.ts):
   - coordinator: Dashboard, Commitments, Queue (badge = pending review
     count), Flows, Team, Evidence;
   - member: My work (dashboard), My tasks (/team), Commitments, Evidence.
2. RequireRole route wrapper in src/components/shell/RequireRole.tsx:
   members opening /queue or /flows are redirected to /team with a short
   notice. Use it in src/App.tsx.
3. "Need more time" (never auto-applied):
   - types (additive): DecisionType adds 'request_extension' and
     'decline_extension'; CommitmentEventType adds 'extension_requested';
     Commitment adds optional extensionRequest { dueAt, by, at, note,
     state: 'pending' | 'approved' | 'declined' }.
   - permissions: the owner (member) can request_extension; only a
     coordinator can decline_extension; approving is a change_deadline
     decision that also marks the pending request approved.
   - applyDecision handles both; a pending request does not change the
     deadline or status, and it does not stop reminders.
   - CommitmentDetail: member form (new date in their time + reason);
     coordinator sees Approve new date / Decline.
4. Team page (coordinator): people grid (name, team, timezone, local time
   at the evaluation clock, open, overdue, done, open blockers) and an
   extension requests inbox.
5. Optional: small approval celebration in the member view using Framer
   Motion, off when prefers-reduced-motion is set.
6. Tests: permissions unit tests; engine decision tests (request keeps the
   deadline, approve changes it and bumps the draft version, member cannot
   decline); UI tests (member nav has no Queue link, /queue redirects);
   e2e: member requests an extension, coordinator approves, deadline and
   draft version change.
```

```
FILES AND MODULES
Modify: src/engine/types.ts, src/engine/permissions.ts, src/engine/history.ts, src/store/actions.ts, src/App.tsx,
src/components/shell/Sidebar.tsx, BottomTabBar.tsx, MobileNavSheet.tsx, src/pages/Team.tsx,
src/pages/CommitmentDetail.tsx, tests/*. Create: src/components/shell/nav.ts,
src/components/shell/RequireRole.tsx.

INTEGRATION REQUIREMENTS
All new decisions go through createActions.decide in src/store/actions.ts, so they are permission-checked and
audit-logged like the existing ones.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e.

COMMITS (push to origin full-integration after each)
show navigation by role
let members request more time
add people grid and extension inbox

ACCEPTANCE CRITERIA
- Members never see Queue or Flows, and are redirected if they type the URL.
- Extension requests never change a deadline until a coordinator approves.
- People grid shows local times at the evaluation clock; all suites pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Switch the top-bar profile to Lena: the sidebar has no Queue or Flows; typing /queue sends you to My tasks.
- As Lena request more time on the press kit; switch to Priya, approve it, and check the Why? timeline.

Next prompt dependency. Needs C08.

C10 Backup, restore, evidence pack and audit filters

Essential Continues: Remaining parts of P13 and P22

Goal. Make local data portable and evidence easy to hand over: full backup and restore, one evidence pack download, audit filters and audited evidence deletion.

Current context. src/store/repo.ts has loadDataset, resetAllData (keeps the audit log), addDecision, addTasks, syncDrafts, putDraft, saveFlow, deleteFlow, addEvidenceFile (SHA-256) and saveRun. src/store/db.ts tables: settings, datasets, decisions, tasks, drafts, flows, audit, evidenceFiles, runs. The Evidence page (src/pages/Evidence.tsx) has tabs Scorecard, Exports, Uploads, Audit log (latest 200, no filters) and Data & settings; its Export data button downloads only the dataset. Evidence files cannot be deleted.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C10 - Backup, restore, evidence pack and audit filters

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
```

```
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Make local data portable and evidence easy to hand over: full backup and restore, one evidence pack download,
audit filters and audited evidence deletion.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/store/repo.ts has loadDataset, resetAllData (keeps the audit log), addDecision, addTasks, syncDrafts,
putDraft, saveFlow, deleteFlow, addEvidenceFile (SHA-256) and saveRun. src/store/db.ts tables: settings,
datasets, decisions, tasks, drafts, flows, audit, evidenceFiles, runs. The Evidence page (src/pages/Evidence.tsx)
has tabs Scorecard, Exports, Uploads, Audit log (latest 200, no filters) and Data & settings; its Export data
button downloads only the dataset. Evidence files cannot be deleted.

TASK: backup and restore, evidence pack, audit filters, audited delete.

1. repo.exportBackup(): JSON { format: 'followthru-backup', version: 1,
   exportedAt, dataset, decisions, tasks, drafts, flows, settings, runs,
   audit, evidenceFiles: metadata only (no blobs) }. Exclude any secret
   setting (for example an AI key added later).
2. Backup schema with Zod in src/schema/backup.ts reusing existing schemas
   where possible; the dataset part must also pass validateDataset.
   repo.importBackup(json) validates, then replaces all tables except
   evidence blobs in one transaction, and writes audit 'data.restore'.
   Coordinator only (or first run with no data).
3. Evidence pack: one JSON download { generatedAt, dataset name, source,
   synthetic flag, evaluation clock, engine mode, flags, scorecard,
   commitments, drafts, audit, evidence file metadata with sha256 }; show
   the SHA-256 of the downloaded content next to the button (use
   repo.sha256Hex).
4. Audit log: filters for actor, action prefix, entity and date range
   (dates shown in UTC and local), "Load more" paging, and a before/after
   view that highlights changed keys. Extend useAuditLog in
   src/store/hooks.ts with a filter argument instead of filtering huge
   arrays in the component.
5. Evidence file deletion: coordinator only, reason required, audit entry
   keeps name, size and sha256.
6. UI: Data & settings tab gets Export backup and Restore backup (file
   input -> validation summary -> confirm Dialog); Exports tab gets
   Download evidence pack.
7. Tests in tests/store.test.ts: export -> reset -> import gives the same
   decisions, tasks, drafts (versions and states) and flows; invalid backup
   rejected with issues; member cannot restore or delete; delete needs a
   reason. UI test for an audit filter.

FILES AND MODULES
Modify: src/store/repo.ts, src/store/actions.ts, src/store/hooks.ts, src/store/index.ts, src/pages/Evidence.tsx,
src/schema/index.ts, tests/store.test.ts, tests/ui.test.tsx. Create: src/schema/backup.ts.
```

```
INTEGRATION REQUIREMENTS
Restore replaces the tables that EngineProvider watches with useLiveQuery, so every screen refreshes
automatically. All writes stay in repo.ts with audit entries.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e.

COMMITS (push to origin full-integration after each)
add full backup and restore
add evidence pack download
add audit filters and file delete

ACCEPTANCE CRITERIA
- Backup and restore round-trip passes in tests and in the browser.
- Evidence pack downloads with a visible SHA-256.
- Audit filters work; evidence deletion needs a reason and is audited.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Approve a draft, export a backup, reset all data, restore the backup: the approval is back.
- Download the evidence pack and open it in a text editor.

Next prompt dependency. Needs C09.

C11 REST API completion and API documentation

Essential Continues: Remaining part of P14

Goal. Finish the API promised in the plan: evidence downloads, CSV input, CORS and written documentation, all reusing the same engine.

Current context. api/_lib/handlers.ts exports handleHealth (GET) and handleProcess (POST { dataset, config? }, JSON only, 2 MB limit, consistent error shape { error: { code, message, issues? } }, logs only status and timing). api/health.ts and api/process.ts wrap them. vite.config.ts has a dev plugin serving /api/health and /api/process. vercel.json rewrites non-API paths to index.html. tests/api.test.ts has 13 tests. There is no /api/evidence, no CORS, no CSV input and no docs/api.md.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C11 - REST API completion and API documentation

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
```

```
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Finish the API promised in the plan: evidence downloads, CSV input, CORS and written documentation, all reusing
the same engine.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
api/_lib/handlers.ts exports handleHealth (GET) and handleProcess (POST { dataset, config? }, JSON only, 2 MB
limit, consistent error shape { error: { code, message, issues? } }, logs only status and timing). api/health.ts
and api/process.ts wrap them. vite.config.ts has a dev plugin serving /api/health and /api/process. vercel.json
rewrites non-API paths to index.html. tests/api.test.ts has 13 tests. There is no /api/evidence, no CORS, no CSV
input and no docs/api.md.

TASK: complete the API.

1. handleProcess also accepts { csv, people, threads? }: build the
   dataset with messagesFromCsv (src/schema/io.ts) and validateDataset.
   If C02 created src/schema/adapter.ts, accept { fixture } and adapt it.
2. handleEvidence (GET): query format=json|csv and
   kind=commitments|drafts|suppressions|scorecard. Data source:
   data/fixture.json when it exists (read with fs at runtime; add
   "functions": { "api/evidence.ts": { "includeFiles": "data/fixture.json" } }
   to vercel.json only if the fixture is committed), otherwise the
   synthetic demo (src/mocks/demoDataset.json) with the response header
   X-FollowThru-Dataset: synthetic-demo and dataset: "synthetic demo
   (not challenge data)" in JSON. Content-Disposition attachment with a
   timestamped file name. 400 for bad parameters, 405 for other methods.
   Use suppressionsToCsv from C07 if it exists; otherwise add it.
3. CORS helper used by all routes: Access-Control-Allow-Origin *,
   methods GET, POST, OPTIONS; OPTIONS returns 204.
4. api/evidence.ts wrapper; add the route and OPTIONS to the dev plugin in
   vite.config.ts.
5. docs/api.md: every endpoint, request and response shapes taken from the
   real types (EngineResult, Scorecard, ValidationIssue), error code table
   (400, 405, 413, 415, 422), curl examples for http://localhost:5173 and
   for LIVE_URL (a clearly marked placeholder), and notes: shadow mode, no
   message text in logs, synthetic fallback header.
6. Tests: tests/api.test.ts for CSV input, evidence kinds and formats,
   bad parameters, OPTIONS; e2e/app.spec.ts checks /api/evidence in the dev
   server.

FILES AND MODULES
Modify: api/_lib/handlers.ts, vite.config.ts, vercel.json (only if fixture committed), tests/api.test.ts,
e2e/app.spec.ts, README.md (link docs/api.md). Create: api/evidence.ts, docs/api.md.

INTEGRATION REQUIREMENTS
Handlers import from src/engine and src/schema directly; no duplicate logic. The dev plugin keeps npm run dev as
```

```
the single launch command.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e, and the curl examples from docs/api.md against npm run
dev.

COMMITS (push to origin full-integration after each)
accept csv input in process api
add evidence api with cors
add api docs

ACCEPTANCE CRITERIA
- /api/health, /api/process (JSON and CSV) and /api/evidence work in dev and tests.
- Synthetic fallback is labelled in headers and body.
- docs/api.md examples were actually run and match the responses.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- With npm run dev running, open http://localhost:5173/api/evidence?format=csv&kind=drafts in the browser: a file downloads.
- Run one curl example from docs/api.md.

Next prompt dependency. Needs C07 (suppressions CSV). Independent of UI prompts.

C12 PWA completion: icons, manifest, install, offline and update

Essential (required for the APK) Continues: Remaining part of P24

Goal. Make FollowThru properly installable and verifiably offline, with PNG icons and manifest fields that PWABuilder needs for the Android package.

Current context. vite.config.ts configures VitePWA (registerType autoUpdate, manifest name FollowThru, start_url /dashboard, a single SVG icon /favicon.svg, workbox globPatterns, navigateFallbackDenylist for /api). public/ contains only favicon.svg. index.html has theme-color and viewport-fit=cover. There are no PNG icons, no apple-touch-icon, no install button, no offline banner and no update prompt. The build precaches 18 entries. Offline behaviour has never been tested.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C12 - PWA completion: icons, manifest, install, offline and update

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
```

```
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Make FollowThru properly installable and verifiably offline, with PNG icons and manifest fields that PWABuilder
needs for the Android package.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
vite.config.ts configures VitePWA (registerType autoUpdate, manifest name FollowThru, start_url /dashboard, a
single SVG icon /favicon.svg, workbox globPatterns, navigateFallbackDenylist for /api). public/ contains only
favicon.svg. index.html has theme-color and viewport-fit=cover. There are no PNG icons, no apple-touch-icon, no
install button, no offline banner and no update prompt. The build precaches 18 entries. Offline behaviour has
never been tested.

TASK: complete and verify the PWA.

1. Original logo public/logo.svg: a check mark merging into a forward arrow,
   violet #7C5CFF to mint #3EE6B0 on #07060D. Generate pwa-192x192.png,
   pwa-512x512.png, maskable-512x512.png (with safe-zone padding) and
   apple-touch-icon-180x180.png with @vite-pwa/assets-generator (dev
   dependency) and a pwa-assets.config.ts; add script "icons". Open the
   generated PNGs to confirm they render.
2. Manifest: name "FollowThru - Chaser Draft Queue", short_name
   "FollowThru", description, start_url "/dashboard", scope "/",
   display "standalone", background_color and theme_color "#07060D",
   orientation "any", categories ["productivity", "business"], icons with
   correct purposes, shortcuts (Queue /queue, Dashboard /dashboard,
   My tasks /team), screenshots: one wide 1280x720 and one narrow 390x844
   PNG in public/screenshots/ captured with Playwright (installed Chrome)
   from the demo data, with form_factor wide and narrow.
3. index.html: apple-touch-icon link, apple-mobile-web-app-capable,
   apple-mobile-web-app-status-bar-style black-translucent.
4. src/components/shell/InstallButton.tsx: beforeinstallprompt on Android
   and desktop; on iOS a Dialog with Share -> Add to Home Screen. Show it in
   the TopBar and on the Landing page.
5. Offline banner in AppShell (navigator.onLine plus online/offline events).
6. Update prompt with useRegisterSW from virtual:pwa-register/react (add
   the vite-plugin-pwa/react types to tsconfig.app.json): "Update available
   - Reload".
7. Verification: add playwright.pwa.config.ts that serves the production
   build with npm run build && npm run preview -- --port 4173 --strictPort,
   script "e2e:pwa", and e2e/pwa.spec.ts: manifest reachable with the
   fields above, icons return 200, a service worker registers, and after
   one visit context.setOffline(true) then reload of /dashboard still
   renders. Run Lighthouse only if it is available locally; never report a
   Lighthouse score you did not run.

FILES AND MODULES
```

```
Modify: vite.config.ts, index.html, package.json, tsconfig.app.json, src/components/shell/TopBar.tsx,
src/components/shell/AppShell.tsx, src/pages/Landing.tsx. Create: public/logo.svg, generated icons,
public/screenshots/*.png, pwa-assets.config.ts, src/components/shell/InstallButton.tsx, playwright.pwa.config.ts,
e2e/pwa.spec.ts.

INTEGRATION REQUIREMENTS
Data already lives in IndexedDB, so the app works offline once the shell is cached. The demo dataset is bundled;
the challenge fixture is bundled only if it is committed.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e, npm run e2e:pwa.

COMMITS (push to origin full-integration after each)
add app logo and icons
complete pwa manifest
add install button and offline banner
add update prompt
add pwa browser test

ACCEPTANCE CRITERIA
- PNG icons (192, 512, maskable 512, apple 180) exist and render.
- e2e:pwa passes: manifest fields, service worker, offline reload.
- Install button works in Chrome desktop; no console errors.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Run npm run build then npm run preview, open http://localhost:4173 in Chrome: an install icon appears in the address bar.
- In Chrome DevTools > Application > Service Workers tick Offline and reload /dashboard: it still opens.

Next prompt dependency. Needs C11 (vercel.json and API settled). Required before C20.

C13 Quality pass: code splitting, error boundaries, accessibility, layout, drive-letter test bug

Essential Continues: P25

Goal. Make every screen robust and accessible on phones and laptops, shrink the main bundle and fix the test failure that happens when the suite starts from a lowercase drive path.

Current context. Main JS chunk 658.69 kB (209.19 kB gzip); 3D is already lazy (SceneCanvas chunk). No route-level code splitting and no error boundaries. Accessibility has not been audited automatically. e2e checks horizontal overflow only on the dashboard at 360 px. Known reproducible bug: when Vitest is started from a lowercase drive path (d:/followthru) every suite fails with 'TypeError: Cannot read properties of undefined (reading config)' and 'no tests'; from D:/followthru all 148 tests pass. VS Code sessions sometimes start in d:↑ollowthru.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C13 - Quality pass: code splitting, error boundaries, accessibility, layout, drive-letter test bug

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
```

```
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Make every screen robust and accessible on phones and laptops, shrink the main bundle and fix the test failure
that happens when the suite starts from a lowercase drive path.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
Main JS chunk 658.69 kB (209.19 kB gzip); 3D is already lazy (SceneCanvas chunk). No route-level code splitting
and no error boundaries. Accessibility has not been audited automatically. e2e checks horizontal overflow only on
the dashboard at 360 px. Known reproducible bug: when Vitest is started from a lowercase drive path
(d:/followthru) every suite fails with 'TypeError: Cannot read properties of undefined (reading config)' and 'no
tests'; from D:/followthru all 148 tests pass. VS Code sessions sometimes start in d:ollowthru.

TASK: quality pass. Fix, do not redesign.

1. Route-level code splitting with React.lazy and Suspense for every page
   except Landing (tests/app.test.tsx asserts the landing synchronously;
   switch other assertions to findBy where needed). Report main chunk
   size before and after (target below 150 kB gzip).
2. src/components/RouteErrorBoundary.tsx around the routes: friendly
   message, Reload, and Reset local data (confirm Dialog, then
   actions.resetAll). Test with a component that throws.
3. Accessibility: add @axe-core/playwright (dev dependency) and
   e2e/a11y.spec.ts that loads the demo as Priya and checks landing, setup,
   profiles, dashboard, commitments, a detail page, queue, flows, team and
   evidence; zero serious or critical violations. Fix labels, contrast,
   landmarks, button names and heading order. Status must never rely on
   colour alone.
4. Responsive sweep in e2e: widths 360, 390, 768, 1024, 1440 over the main
   pages with no horizontal overflow; fix layouts that fail.
5. Reduced motion: an e2e test with reducedMotion 'reduce' confirms the
   landing renders Fallback2D and nothing animates.
6. Drive-letter bug: reproduce it first with
   node -e "require('child_process').spawnSync('npx vitest run',
{cwd:'d:/followthru',shell:true,stdio:'inherit'})"
   (all suites fail) and the same command with cwd 'D:/followthru' (all
   pass). Find the root cause in how Vitest resolves the config and the
   projects (engine, ui) in vitest.config.ts when the drive letter case
   differs, and fix it so npm test passes from both paths, for example by
   normalising the root in vitest.config.ts. Check the Vitest changelog or
   issues for this behaviour instead of guessing. Add a check (a small
   script, for example scripts/check-drive-case.cjs run by an npm script)
   that runs the suite with a lowercase cwd, and record the cause and fix
   in docs/integration-status.md. Never hide the failure by skipping tests.
7. Remove dead code, console.log calls and TODOs.
```

```
FILES AND MODULES
Modify: src/App.tsx, tests/app.test.tsx, tests/ui.test.tsx, vitest.config.ts (only if the root cause needs it),
pages with a11y issues, docs/integration-status.md, package.json. Create: src/components/RouteErrorBoundary.tsx,
e2e/a11y.spec.ts.

INTEGRATION REQUIREMENTS
Lazy routes keep the EngineProvider at the top of App, so store state survives navigation.

TESTING REQUIREMENTS
npm test (10-run loop), npm run typecheck, npm run build (paste chunk sizes), npm run e2e including the new
specs.

COMMITS (push to origin full-integration after each)
split routes into lazy chunks
add route error boundary
fix accessibility issues
fix layouts at small widths
run tests from any drive letter case

ACCEPTANCE CRITERIA
- Main chunk smaller than before, sizes reported.
- axe: zero serious or critical violations on all listed pages.
- No horizontal overflow at the five widths.
- npm test passes from both d:/followthru and D:/followthru; all suites pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open the app on your phone (npm run dev -- --host, use the Network URL) and click through as coordinator and member.
- Read the chunk sizes and the drive-letter fix in Claude's report.

Next prompt dependency. Needs C12. Run after the feature prompts so it checks the final screens.

C14 Commitment detail: plain-English Why, time travel and thread view

Recommended Continues: Remaining part of P19

Goal. Explain each decision in plain sentences and let a viewer scrub through time to watch owner, deadline, status and draft change.

Current context. src/pages/CommitmentDetail.tsx shows facts, an evidence timeline (event type, time, person, message id, thread, rule, quote), linked messages, submissions and blockers, the draft card and role actions. There are no plain-English reasons, no time travel and no full-thread view.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C14 - Commitment detail: plain-English Why, time travel and thread view

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.
```

```
GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Explain each decision in plain sentences and let a viewer scrub through time to watch owner, deadline, status and
draft change.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/pages/CommitmentDetail.tsx shows facts, an evidence timeline (event type, time, person, message id, thread,
rule, quote), linked messages, submissions and blockers, the draft card and role actions. There are no plain-
English reasons, no time travel and no full-thread view.

TASK: explainability upgrades.

1. src/components/domain/explain.ts, pure: explain(commitment, draft,
   people, threads) returns sentences with message ids and rule names, for
   example "Owner is Lena because Amir handed it over (m_07) and Lena
   accepted (m_08)." and "No reminder: completed (m_10, linked from another
   thread by keyword overlap)." Unit tests for each event kind.
2. Time travel: a slider over the sorted unique times of the commitment's
   linked messages, decisions and the evaluation clock. For position t,
   call runEngine with messages sentAt <= t, decisions at <= t, manual
   tasks createdAt <= t, now = t and no previous drafts (say on screen
   that drafts in this view are recomputed, not the stored queue). Memoise
   per position and debounce 150 ms. Show status, owner, deadline and draft
   state at t. Step buttons always; a play button that is disabled when
   reduced motion is on. Put the helper in src/components/domain/
   timeTravel.ts; keep src/engine unchanged.
3. Thread view: clicking a message id opens the Dialog (sheet variant) with
   the whole thread in order, the message highlighted, times in the
   author's timezone and the viewer's timezone.
4. Tests: explain unit tests; time travel on the demo commitment c_m_03
   (open before m_10, done at m_10); a UI test that moving the slider
   changes the status chip; an e2e step that uses the slider.

FILES AND MODULES
Modify: src/pages/CommitmentDetail.tsx, tests/ui.test.tsx, e2e/app.spec.ts. Create:
src/components/domain/explain.ts, src/components/domain/timeTravel.ts, tests/explain.test.ts.

INTEGRATION REQUIREMENTS
Time travel reuses runEngine directly, so it always matches the real engine; it never writes to the store.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e.

COMMITS (push to origin full-integration after each)
add plain english why panel
add time travel slider
add thread view sheet

ACCEPTANCE CRITERIA
```

```
- Every commitment shows plain-English reasons with message ids.
- Slider shows the deadline-to-done story for c_m_03; thread sheet opens.
- Engine untouched; all suites pass.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open /commitments/c_m_03 and drag the slider from start to end: status goes from open to done.
- Click a message id and read the thread sheet.

Next prompt dependency. Needs C13 (lazy routes in place). Optional to skip if time is short.

C15 Landing page scroll story

Optional enhancement (cut first) Continues: P16

Goal. Replace the minimal hero with the planned scroll story, using real counts and keeping a complete fallback.

Current context. src/pages/Landing.tsx shows a headline, subtitle, an Open app link to /setup and <Stage> (src/three/Stage.tsx) which lazy-loads SceneCanvas.tsx (one wireframe mesh) and falls back to Fallback2D.tsx without WebGL or with reduced motion. Lenis is installed but unused. Design tokens are in src/styles/tokens.ts and global.css.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C15 - Landing page scroll story

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.
```

```
GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Replace the minimal hero with the planned scroll story, using real counts and keeping a complete fallback.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
src/pages/Landing.tsx shows a headline, subtitle, an Open app link to /setup and <Stage> (src/three/Stage.tsx)
which lazy-loads SceneCanvas.tsx (one wireframe mesh) and falls back to Fallback2D.tsx without WebGL or with
reduced motion. Lenis is installed but unused. Design tokens are in src/styles/tokens.ts and global.css.

TASK: a lightweight, original scroll story. Do not copy any site's code or
assets.

1. One pinned canvas behind HTML sections: hero ("Chase what's late. /
   Never what's done."), chaos (message chips tangled), scan (commitments
   light up), sorted (chips orbit in status colours done mint, overdue
   coral, tentative amber, deferred sky), queue (overdue chips stack into
   cards, done chips fade with a suppressed pulse), edge cases (six cards:
   maybe Friday, timezones, delegation, completion elsewhere, silence
   versus not done, deadline moved), CTA (Open app, View evidence).
2. Instanced mesh: at most 320 chips on desktop and 120 on phones; per-
   section targets precomputed; useFrame lerps positions (no React state
   per frame); pause when the tab is hidden; frameloop demand when idle.
3. Counts come from the real engine via useEngine when data is loaded,
   labelled with the dataset name (and "synthetic" for the demo); without
   data show no numbers.
4. Lenis only on the landing page, off with reduced motion. Fallback2D and
   CSS-only sections when WebGL is missing or motion is reduced.
5. Open app goes to /dashboard when data and a profile exist, otherwise
   /setup.
6. Tests: tests/app.test.tsx landing assertions still pass; e2e checks no
   console errors and the reduced-motion fallback. Keep the 3D code in a
   lazy chunk and report its size.

FILES AND MODULES
Modify: src/pages/Landing.tsx, src/three/SceneCanvas.tsx, tests/app.test.tsx, e2e/app.spec.ts. Create:
src/three/StoryScene.tsx and small helpers as needed.

INTEGRATION REQUIREMENTS
Only the landing route changes; the app shell and data screens are untouched.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build (3D chunk size), npm run e2e.

COMMITS (push to origin full-integration after each)
add landing story sections
add 3d story scene
add landing fallback

ACCEPTANCE CRITERIA
```

```
- Story renders smoothly on a laptop and degrades to the fallback with reduced motion.
- Counts match the dashboard for the loaded data; no console errors.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Scroll the landing page on your laptop and phone.
- Turn on reduced motion in your OS and confirm the static version appears.

Next prompt dependency. Needs C13. Safe to skip entirely.

C16 Optional AI-assisted extraction with a labelled rules fallback

Optional (needs your own API key) Continues: P23

Goal. Add an opt-in AI pre-pass that can only add suggestions or review flags, never decide status, with an honest fallback label.

Current context. EngineFlags.ai exists; modeLabel in src/engine/run.ts returns 'Rules (fallback)' when ai is true because no provider exists. The Data & settings tab explains this. C05 adds src/engine/extractor.ts (Extractor interface, rulesExtractor).

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C16 - Optional AI-assisted extraction with a labelled rules fallback

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.
```

```
GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Add an opt-in AI pre-pass that can only add suggestions or review flags, never decide status, with an honest
fallback label.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
EngineFlags.ai exists; modeLabel in src/engine/run.ts returns 'Rules (fallback)' when ai is true because no
provider exists. The Data & settings tab explains this. C05 adds src/engine/extractor.ts (Extractor interface,
rulesExtractor).

TASK: optional Gemini-assisted signal suggestions. OFF by default.

1. Check the current Gemini API documentation for the endpoint and a
   current free-tier flash model name; make the model a setting. Do not
   guess names from memory.
2. The key is entered on the Data & settings tab, stored only in IndexedDB
   settings, excluded from backups, exports, audit entries and the API,
   and never committed.
3. AI pre-pass runs in the browser (src/store, async), per thread, asking
   for JSON only, validated with Zod: [{ messageId, kind, hedged,
   refersToMessageId?, confidence }]. Invalid JSON -> fallback.
4. Merge rule, deterministic and pure: EngineInput gains optional
   aiSignals: Record<string, Signal[]> (additive). The engine accepts an AI
   suggestion only when the rules agree; disagreement or confidence below
   0.7 adds review reason 'ai_disagreement' (additive ReviewReason) and
   never changes status. Deadlines still go through parseDeadline.
5. Fallback with a visible reason for: no key, offline, timeout (8 s),
   HTTP 429, error, invalid output -> mode "Rules (fallback)". Success ->
   "AI assisted". Show the mode in the top bar and on the Evidence page.
6. Official evidence (npm run evidence) always runs rules-only; say so in
   the scorecard header.
7. Tests with mocked fetch: valid output merges, invalid output falls back,
   timeout falls back, low confidence goes to review, the key never
   appears in exportBackup output.

FILES AND MODULES
Create: src/store/ai.ts, tests/ai.test.ts. Modify: src/engine/types.ts, src/engine/run.ts, src/engine/history.ts,
src/store/EngineProvider.tsx, src/pages/Evidence.tsx, src/components/shell/TopBar.tsx, scripts/evidence.ts.

INTEGRATION REQUIREMENTS
The engine stays pure: all network work happens in the store before runEngine. Without a key the app is
unchanged.

TESTING REQUIREMENTS
npm test, npm run typecheck, npm run build, npm run e2e (must pass without a key).

COMMITS (push to origin full-integration after each)
add optional ai suggestions
add ai fallback labels
```

```
add ai tests

ACCEPTANCE CRITERIA
- With no key the top bar shows Rules and everything works.
- With a key, suggestions never change status on their own; fallbacks are labelled.
- The key never leaves IndexedDB; tests prove it is excluded from backups.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Without a key: top bar says Rules.
- Optional: add your own key, reload, and check the mode label and the review reasons.

Next prompt dependency. Needs C05 and C10. Skip if you have no key or no time.

C17 Screenshot evidence and project documentation

Essential Continues: P26

Goal. Produce timestamped screenshots as evidence and the documents a judge reads, with honest placeholders for anything only you can supply.

Current context. README.md has run commands, data sources and an architecture table. docs/ has integration-status.md, fixture-notes.md, track-a-status.md, track-b-status.md and planning documents. e2e/app.spec.ts saves four screenshots to test-results/screens/ (gitignored). There is no screenshots script, no evidence/screenshots folder and no architecture, pilot plan, growth experiment or business case documents.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C17 - Screenshot evidence and project documentation

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
```

```
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Produce timestamped screenshots as evidence and the documents a judge reads, with honest placeholders for
anything only you can supply.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
README.md has run commands, data sources and an architecture table. docs/ has integration-status.md, fixture-
notes.md, track-a-status.md, track-b-status.md and planning documents. e2e/app.spec.ts saves four screenshots to
test-results/screens/ (gitignored). There is no screenshots script, no evidence/screenshots folder and no
architecture, pilot plan, growth experiment or business case documents.

TASK: screenshots plus documentation.

1. scripts/screenshots.ts using @playwright/test's chromium with channel
   'chrome' (installed Chrome; no download). Start npm run preview on port
   4173 (build first) or reuse a running server. Load the demo data, or the
   challenge fixture when data/fixture.json exists. Capture at 1440x900 and
   390x844: landing, dashboard, commitments, detail of a completed-in-
   another-thread commitment (demo c_m_03), queue with recipient review,
   suppressed lane, flows, team as coordinator, member workspace as Lena,
   evidence scorecard, audit log. Save to evidence/screenshots/<timestamp>/
   with index.md listing file, time, page and what it proves, and the data
   source (SYNTHETIC or challenge fixture). Add script "screenshots".
   Commit only the latest set.
2. README.md rewrite: banner screenshot, tagline, LIVE_URL, APK_URL and
   DEMO_VIDEO_URL as clearly marked placeholders, the problem in three
   lines, features, edge cases table (case -> rule in code -> test file ->
   evidence), scoring table (requirement -> where -> proof file),
   architecture diagram in Mermaid, stack with one-line reasons, data flow,
   folder structure, quick start with the real scripts, API docs link,
   product change summary, honest limitations (synthetic demo data, fixture
   status, shadow mode only, local demo profiles are not security, not
   production ready), roadmap, team credits with TEAMMATE_1 and TEAMMATE_2
   placeholders (never invent names).
3. docs/architecture.md: modules, pure engine design, data model, commitment
   state machine (Mermaid), idempotency and versioning, freshness check,
   timezone strategy, why IndexedDB instead of an external database.
4. docs/pilot-plan.md: shadow a coordinator; metrics false reminder rate,
   overdue-action coverage, approval rate with formulas; thresholds before
   controlled sends (false reminders under 5 percent, coverage at least 90
   percent, approval at least 80 percent over two weeks); go or no-go table;
   coordinator feedback template.
5. docs/growth-experiment.md: a table for ten overdue commitments with
   columns id, who was approached (role), date, channel, draft shown,
   response, acted within 48 h, useful (definition), notes, plus
   Observed versus Assumption columns. Leave every row EMPTY for me. Never
   invent results.
6. docs/business-case.md: a short placeholder that says where I will add
```

```
   the business case PDF and pitch deck; do not write business results.

FILES AND MODULES
Create: scripts/screenshots.ts, evidence/screenshots/<timestamp>/*, docs/architecture.md, docs/pilot-plan.md,
docs/growth-experiment.md, docs/business-case.md. Modify: README.md, package.json.

INTEGRATION REQUIREMENTS
Screenshots come from the real running app with real engine output; nothing is mocked or edited.

TESTING REQUIREMENTS
npm run screenshots (open several images), npm test, npm run typecheck, npm run build. Check Mermaid blocks
render on GitHub.

COMMITS (push to origin full-integration after each)
add screenshot script
save screenshot evidence
update readme for judges
add architecture and pilot docs
add growth experiment template

ACCEPTANCE CRITERIA
- evidence/screenshots/<timestamp>/ with index.md and both sizes.
- README and docs are factual, with placeholders clearly marked.
- No invented results, names or URLs anywhere.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Open evidence/screenshots/<timestamp>/index.md and a few images.
- Read README.md on GitHub; search it for LIVE_URL to see the placeholders.
- Fill docs/growth-experiment.md with your real experiment rows when you have them.

Next prompt dependency. Needs C13 (and C14/C15 if you ran them, so screenshots show the final screens).

C18 Final QA and release check

Essential Continues: P27

Goal. Verify the whole brief end to end, report PASS, FAIL or BLOCKED with proof paths, and fix only small issues.

Current context. Use whatever scripts exist by now (check package.json): test, typecheck, build, e2e, evidence, plus e2e:pwa and screenshots if C12 and C17 ran.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C18 - Final QA and release check

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
```

```
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Verify the whole brief end to end, report PASS, FAIL or BLOCKED with proof paths, and fix only small issues.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
Use whatever scripts exist by now (check package.json): test, typecheck, build, e2e, evidence, plus e2e:pwa and
screenshots if C12 and C17 ran.

TASK: release check. Report first, then fix small issues only. Anything
larger goes into the report as FAIL with a proposed prompt.

1. Run every available script: npm test (twice), npm run typecheck,
   npm run build, npm run e2e, npm run e2e:pwa (if present),
   npm run evidence -- --label release (fixture if present, else --demo and
   mark the scoring line BLOCKED), npm run screenshots (if present).
2. Write docs/release-check.md with PASS, FAIL or BLOCKED and a proof path
   for each item:
   - ten labelled commitments correct with evidence ids (BLOCKED without
     the fixture);
   - zero active reminders for completed, cancelled or deferred work;
   - one action one draft; a deadline update changes the same draft;
   - maybe Friday tentative, timezones, delegated owner, completion in
     another thread, no response versus not completed;
   - silence never changes status (response pending shown);
   - stale-message suppression (completion after drafting) and recipient
     review blocking approval until confirmed;
   - starting state, negotiated scope and final evidence saved;
   - validation messages for bad input; JSON and CSV import and export;
   - readable evidence: scorecard, screenshots, test report, timestamps,
     hashes;
   - optional AI labelled and rules fallback with no key;
   - coordinator and member flows with permissions; audit log;
   - PWA installable, offline reload, phone layout;
   - API health, process, evidence;
   - README honest limitations; growth experiment and pilot plan present
     (rows may be empty until I fill them).
3. Hygiene: git grep for secrets (key=, apiKey, AIza, BEGIN PRIVATE KEY,
   .env), check .gitignore covers .env*, *.keystore, *.jks, *.apk, *.aab,
   signing-key-info.txt; confirm no console.log or TODO in src; check
   git log --format=%an%n%B for co-author or AI mentions on this branch;
   docs/TRACK_A_BACKEND_PROMPT.md still untracked and untouched.
4. Commit the release evidence run and docs/release-check.md.

FILES AND MODULES
Create: docs/release-check.md, evidence/runs/*_release (or demo-runs if synthetic, then copy the scorecard
summary into release-check.md). Modify: .gitignore if needed.

INTEGRATION REQUIREMENTS
No feature work here; this prompt only verifies and documents.

TESTING REQUIREMENTS
Everything listed in step 1, with real output summaries in the report.
```

```
COMMITS (push to origin full-integration after each)
save release evidence
add release check
remove unused files (only if any)

ACCEPTANCE CRITERIA
- docs/release-check.md exists, every item has PASS, FAIL or BLOCKED with proof.
- All scripts pass, or failures are reported honestly with next steps.
- No secrets or signing files tracked.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Read docs/release-check.md. Every BLOCKED line should name what is missing (for example the fixture).
- Decide whether to proceed to C19 (merge and deploy).

Next prompt dependency. Needs all essential prompts that you chose to run (at least C01, C04 to C13 and C17).

C19 Merge into main and deploy on Vercel

Essential (needs your authorisation and Vercel account) Continues: MERGE prompt and 'Deploy on Vercel' page

Goal. Open a pull request from full-integration, merge it only after your explicit go-ahead, deploy from main, and record the real live URL.

Current context. main on GitHub is at 3fd93f3 (frontend phase 1 merged); full-integration contains all of it plus the backend, integration and later work. Previous PRs #1 to #4 were merged with merge commits. vercel.json sets the build command, output dist and the SPA rewrite. The demo data option only appears in production builds when VITE_DEMO_MODE=true.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C19 - Merge into main and deploy on Vercel

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
```

```
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Open a pull request from full-integration, merge it only after your explicit go-ahead, deploy from main, and
record the real live URL.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
main on GitHub is at 3fd93f3 (frontend phase 1 merged); full-integration contains all of it plus the backend,
integration and later work. Previous PRs #1 to #4 were merged with merge commits. vercel.json sets the build
command, output dist and the SPA rewrite. The demo data option only appears in production builds when
VITE_DEMO_MODE=true.

TASK, PART A (you may do this now): prepare the pull request.
1. Confirm docs/release-check.md exists and npm test, typecheck, build and
   e2e pass on full-integration, and that it is pushed (local and
   origin/full-integration at the same commit).
2. git fetch origin and confirm full-integration contains origin/main
   (git merge-base --is-ancestor origin/main full-integration). If not,
   STOP and report; do not rebase.
3. Create a pull request with gh pr create --base main --head
   full-integration. Title: lowercase, no symbols, for example
   "complete followthru app". Body: plain summary of what changed and the
   release check link; no AI mention. Print the PR URL and STOP.

PART B (only when I reply exactly: merge into main):
4. Re-run npm test and npm run build, then merge with
   gh pr merge <number> --merge (merge commit, no squash, no force, keep
   the branch). Verify origin/main now equals the merge commit and that
   a fresh clone of main builds (git worktree add in a temporary folder
   outside the repo, npm ci, npm run build, then remove that worktree).

PART C (after I paste the Vercel URL):
5. Verify the URL without trusting it blindly: fetch /api/health (expect
   ok true), /, /manifest.webmanifest, and run the e2e smoke journey
   against it if practical (PLAYWRIGHT base URL override). Report the real
   results.
6. Replace LIVE_URL in README.md and docs/api.md with the URL, commit
   "add live demo link" on full-integration, push, and open a small PR to
   main (merge only after I say merge into main again).
Never invent a URL. If verification fails, report it and change nothing.

FILES AND MODULES
Modify (Part C): README.md, docs/api.md.

INTEGRATION REQUIREMENTS
Vercel builds from main using vercel.json; the API functions in api/ deploy automatically. If the challenge
fixture is not committed, the deployed app needs VITE_DEMO_MODE=true or a user import to have any data.

TESTING REQUIREMENTS
```

```
Part A: full local suite. Part B: fresh-clone build. Part C: live checks with real output.

COMMITS (push to origin full-integration after each)
add live demo link (Part C only)

ACCEPTANCE CRITERIA
- PR created and merged only after the exact phrase.
- main builds from a fresh clone.
- Live URL verified (health, landing, manifest) and recorded in README.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- Vercel (you): vercel.com -> sign in with GitHub -> Add New -> Project -> import shivv-14/followthru.
- Framework preset Vite, build command npm run build, output dist, production branch main.
- Environment variable VITE_DEMO_MODE=true if the challenge fixture is not committed (so judges can load the labelled synthetic demo).
- Deploy, then open <your-url>/api/health and paste the URL into Claude for Part C.

Next prompt dependency. Needs C18 with no unexplained FAIL. Part B needs your exact phrase; Part C needs your Vercel URL.

C20 Android APK with PWABuilder and asset links

Optional or required by your submission (needs your accounts and phone) Continues: 'Android APK with PWABuilder' page

Goal. Package the deployed PWA as an Android APK, remove the browser bar with Digital Asset Links, and publish the APK without ever committing signing keys.

Current context. C12 provides the PNG icons, manifest and service worker PWABuilder checks. C19 provides the live HTTPS URL. vercel.json rewrites every non-API path to index.html, which would also catch /.well-known/assetlinks.json unless excluded.

COPY-PASTE BLOCK FOR CLAUDE CODE:

```
PROMPT C20 - Android APK with PWABuilder and asset links

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in the repository D:\followthru.
Single-laptop workflow: all work happens on this laptop. There is no second
laptop, no parallel branch and no Track A / Track B split any more.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work on branch full-integration. If you are on another branch, run
   git checkout full-integration and git pull origin full-integration.
   If git status shows uncommitted changes that you did not make in this
   session, STOP and show them to me. Never stash, reset or discard them.
   The untracked file docs/TRACK_A_BACKEND_PROMPT.md is mine: leave it
   untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie
  (IndexedDB), Zod 4, Vitest 5 + Testing Library + fake-indexeddb,
  Playwright (uses the installed Chrome, channel "chrome"),
  vite-plugin-pwa, Vercel functions in api/.
- package.json scripts: dev, build, preview, test, test:watch, typecheck,
  evidence, e2e (plus any added by earlier continuation prompts - check).
- src/engine/types.ts is the single domain contract. Never create
  duplicate domain types elsewhere. Changes must be additive (new optional
  fields, new union members, new types) unless I approve a breaking change.
  tests/contract.test.ts locks the engine's public exports: update it on
  purpose and log every contract change in docs/integration-status.md.
- The engine (src/engine/) is pure and deterministic. Never call
  Date.now() or new Date() inside src/engine. The evaluation clock is
  EngineConfig.now. Entry point: runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts
  (resolveCommitments) -> drafts.ts (buildDrafts) -> approval.ts ->
  evaluate.ts. Flows in flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, the
  requester or a coordinator can close work; weak or ambiguous evidence
  sets needsReview and never changes status silently or triggers a chase;
  deadlines are read in the speaker's timezone and stored in UTC; one draft
  per commitment (id d_<commitmentId>) and updates bump its version;
  done, cancelled and deferred work keeps zero active drafts; approval
  re-checks freshness and needs a confirmed recipient; shadow mode only,
  nothing is ever sent.
- The challenge fixture (Starter_Data/C14/fixture.json) is NOT in the repo
  unless data/fixture.json exists now. Never invent labels, fixture fields
  or accuracy numbers. Synthetic data (src/mocks/demoDataset.json,
  tests/fixtures/*.json) must always be labelled synthetic and never be
```

```
  presented as challenge evidence.

GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test,
  npm run typecheck and npm run build; all must pass. Review
  git diff --staged before committing.
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols
  or punctuation, no prefixes such as feat or fix with a colon, one line,
  git commit -m "message". Never add Co-Authored-By or any mention of AI or
  Claude in commits, code, docs or PR text. Never change git config.
- Stage files by explicit path. Never use git add -A or git add . and never
  commit node_modules, dist, .env files, API keys, keystores, APK files or
  personal data.
- Never force push, never run git reset --hard, git clean, git checkout -- .
  or rebase a pushed branch, never delete branches. Never merge into main
  unless I write exactly: merge into main.
- After committing, push with: git push origin full-integration
- Report honestly: summarise the real command output. Never claim a test,
  build, deploy, score or screenshot that you did not actually produce.
- Do not add a test that only passes by weakening an existing test. Never
  delete or skip a failing test to make the suite green.

GOAL
Package the deployed PWA as an Android APK, remove the browser bar with Digital Asset Links, and publish the APK
without ever committing signing keys.

CURRENT CONTEXT (verified on 17 Oct 2026 at commit cad765d; re-check it
against the code before relying on it)
C12 provides the PNG icons, manifest and service worker PWABuilder checks. C19 provides the live HTTPS URL.
vercel.json rewrites every non-API path to index.html, which would also catch /.well-known/assetlinks.json unless
excluded.

TASK, PART A (before you use PWABuilder):
1. Add to .gitignore: *.keystore, *.jks, signing-key-info.txt, *.apk,
   *.aab, and a pwabuilder-output/ folder. Commit "ignore android signing
   files".
2. Update vercel.json so /.well-known/(.*) is NOT rewritten to index.html
   and is served with Content-Type application/json. Add a test or a
   documented curl check. Commit "serve asset links file".

PART B (after I paste the assetlinks.json content from the PWABuilder zip):
3. Create public/.well-known/assetlinks.json with exactly that content
   (validate it is JSON). Commit "add asset links", push, PR to main, merge
   only after I write merge into main. After the deploy, fetch
   <LIVE_URL>/.well-known/assetlinks.json and confirm it returns the JSON.

PART C (after I give you the local path of the .apk and say: publish release):
4. Never read, copy or commit the keystore or signing-key-info.txt.
5. Create a GitHub release with gh release create v1.0.0 <path-to-apk>
   --title "FollowThru 1.0.0" --notes "<plain notes, no AI mention>"
   --target main. Print the release URL from gh's output.
6. Replace APK_URL in README.md with the release download URL, commit
   "add apk download link", push, PR to main as above.

FILES AND MODULES
Modify: .gitignore, vercel.json, README.md. Create: public/.well-known/assetlinks.json (content supplied by you).

INTEGRATION REQUIREMENTS
The APK is a Trusted Web Activity wrapper around the live site, so every app update deploys through Vercel
without a new APK.

TESTING REQUIREMENTS
Part A: npm run build and the curl check against npm run preview. Part B: live fetch of assetlinks.json. Part C:
open the release URL.

COMMITS (push to origin full-integration after each)
ignore android signing files
serve asset links file
add asset links
add apk download link
```

```
ACCEPTANCE CRITERIA
- Keystore and signing info never tracked (git ls-files shows none).
- assetlinks.json served as JSON from the live site.
- GitHub release exists with the APK; README links to it.

FINAL REPORT
When done, report: the files changed, the real results of every command
you ran (test counts, typecheck, build sizes, e2e), the commits with
hashes, and anything not done with the reason. Then stop and wait.
```

You check

- PWABuilder (you): open pwabuilder.com, enter the live URL, Start, fix any red items, then Package for stores -> Android -> Generate.
- Unzip outside the repo (for example D:\followthru-android\). Keep signing.keystore and signing-key-info.txt in a safe place, never in the repo.
- Give Claude the assetlinks.json content (Part B), then the APK path and the phrase publish release (Part C).
- Install the APK from the release link on an Android phone and open it: no browser bar after asset links are live.

Next prompt dependency. Needs C12 and C19 (live HTTPS URL).

6. Reusable operational prompts

These are adapted to this repository and its real scripts. Replace the parts in angle brackets.

CONTINUE Resume after a new Claude Code session

Use: After a usage limit, a crash, or any new chat.

```
Read CLAUDE.md, README.md, docs/integration-status.md and, if they exist,
docs/release-check.md and docs/product-change.md.
Run: git status --short ; git branch --show-current ; git log --oneline -15 ;
git fetch origin ; git status -sb
We work on branch full-integration only (single laptop). If git status
shows uncommitted changes, list them and ask me before touching them;
never stash, reset or discard. Leave docs/TRACK_A_BACKEND_PROMPT.md
untracked.
We are following the FollowThru Continuation Prompt Playbook
(docs/FollowThru_Continuation_Prompt_Playbook.pdf). The last prompt I ran
was <C-number>. Check that its acceptance criteria are met by inspecting
the code and running npm test, npm run typecheck and npm run build (and
npm run e2e if the prompt touched the UI). Report what is complete, what
is missing and what failed, with real output. If something from that
prompt is missing, finish only that part with small commits (lowercase,
3-8 words, no symbols, no AI mention), push to origin full-integration,
then wait for my next prompt. Never merge into main unless I write
exactly: merge into main.
```

FIX Reproduce and repair a specific bug

Use: Whenever something breaks in a command or a screen.

```
Read CLAUDE.md first. Branch: full-integration. Run git status --short and
do not touch uncommitted work you did not make.
PROBLEM: <one or two lines describing what you saw>
ERROR (exact text or screenshot description): <paste>
WHERE: <command or page, for example npm test, /queue on phone, profile Lena>
TASK:
1. Reproduce it first (a failing test, a script run or a Playwright step).
   If you cannot reproduce it, say so and show what you tried.
2. Find the root cause and explain it in two or three lines, naming the
   file and function (for example src/engine/history.ts findTarget).
3. Fix the cause, not the symptom. Keep the engine pure and the contract
   additive. Never delete, skip or weaken a test to make it pass.
4. Add or update a test that fails before the fix and passes after.
5. Run npm test, npm run typecheck, npm run build, and npm run e2e if the
   UI is involved.
6. Review git diff --staged, commit with a short lowercase message such as
   "fix stale draft check before approval", push to origin
   full-integration, and report the before and after output.
```

SCORECARD Run the evaluation and report actual results

Use: Any time you want the real numbers.

```
Read CLAUDE.md first. Do not change any code in this prompt.
1. If data/fixture.json exists run npm run evidence (add -- --label check
   if that option exists in scripts/evidence.ts); otherwise run
   npm run evidence -- --demo and state clearly that the data is SYNTHETIC
   and the owner/deadline/status check is not scored.
2. Open the newest scorecard.md in evidence/ and report exactly:
   - labelled commitments correct: x of y (or "not scored: no labels");
   - active reminders on completed, cancelled or deferred work: n;
   - duplicate drafts: n; re-run identical: yes or no;
   - each failing labelled item with expected versus actual and evidence
     message ids;
   - the evidence folder path and its manifest hashes.
3. If anything fails, diagnose the likely cause (file and rule) and propose
   the smallest general fix, but do not edit code until I say go.
```

INTEGRATION CHECK Confirm the UI is wired to the real engine and store

Use: After UI work, or before a demo.

```
Read CLAUDE.md first. Do not change code unless I approve the findings.
Audit whether every screen and action uses real engine and store
functionality rather than placeholders or mocks.
1. List every route in src/App.tsx and, for each page in src/pages, the
   data it reads (useEngine fields, store hooks) and the actions it calls
   (useActions functions). Flag any hardcoded arrays of domain data, any
   import from src/mocks other than demoDataset.json in Setup, any button
   without a handler, any "coming soon" or placeholder text, and any
   component importing Dexie or src/store/db.ts directly.
2. For each action in src/store/actions.ts confirm: permission check,
   repo write, audit entry, and a test that covers it (grep tests/).
3. grep src for TODO, FIXME, console.log, Math.random used for data, and
   Date.now inside src/engine.
4. Run npm run e2e and npm test and report results.
5. Output a table: screen or action, data source, real or placeholder,
   test coverage, issue. Then propose fixes as FIX prompts, ordered by
   impact. Wait for my go before changing anything.
```

RELEASE CHECK Verify the full acceptance checklist before a release

Use: Before merging into main, deploying or submitting.

```
Read CLAUDE.md first. Branch full-integration, clean tree required (show
git status; ask me if it is not clean). Report first; fix only trivial
issues; anything larger is reported as FAIL.
1. Run all scripts that exist in package.json: test (twice), typecheck,
   build, e2e, e2e:pwa, evidence (fixture if present, else --demo labelled
   SYNTHETIC), screenshots.
2. Produce PASS, FAIL or BLOCKED with a proof path for: ten labelled
   commitments; zero reminders on closed work; one action one draft with
   versioned deadline updates; the five edge cases; silence never changes
   status; stale suppression and recipient review; starting state, scope
   and final evidence; validation of bad input; JSON and CSV import and
   export; readable timestamped evidence with hashes; AI labelled or
   absent with rules fallback; coordinator and member flows with
   permissions and audit; PWA install and offline; API endpoints; README
   limitations; no secrets, keystores or APKs tracked; no co-author or AI
   mentions in this branch's commits.
3. Update docs/release-check.md with the results and the date, commit
   "update release check", push to origin full-integration. Do not merge
   into main unless I write exactly: merge into main.
```

BLOCKER REPORT

List missing external inputs and keep independent work moving

Use: When you are waiting on files, accounts or decisions.

```
Read CLAUDE.md, docs/integration-status.md and docs/release-check.md (if
present). Do not invent any missing input.
1. Check and report the status of each external dependency:
   - data/fixture.json (challenge fixture) present? committed?
   - official submission deadline and demo format confirmed?
   - Vercel project and live URL (look for LIVE_URL placeholders)?
   - PWABuilder output, assetlinks.json, APK release (APK_URL)?
   - optional Gemini key: never ask me to paste it into the chat; you
     cannot see whether one is set in the app unless I tell you.
   - business case PDF, pitch deck, demo video (DEMO_VIDEO_URL), teammate
     names (TEAMMATE_1, TEAMMATE_2), real growth-experiment rows?
2. For each missing item: what is blocked, who can provide it (me or the
   organisers), and the exact action needed.
3. List the playbook prompts (C01 to C20) that can still run without these
   inputs, in dependency order, and the first one to run next.
4. Then continue with that first unblocked prompt only if I reply go.
```

7. What Claude does and what you do

Action	Claude (locally)	You
Code, tests, docs, evidence runs, screenshots	Yes, C01-C18	Run the "You check" steps
Copy the challenge fixture into `data/`	No	Yes, before C02
Commit the fixture	Only after you say yes	Decide (check the challenge terms)
Pull request to main	Creates it (C19 Part A)	Review it
Merge into main	Only after the exact phrase *merge into main*	Authorise
Vercel project and deploy	No	Import repo, preset Vite, output dist, set VITE_DEMO_MODE=true if the fixture is not committed
Verify the live URL and record it	Yes, after you paste it (C19 Part C)	Paste the real URL
PWABuilder package	No	Generate on pwabuilder.com, keep the keystore outside the repo
assetlinks.json	Adds the file you paste (C20 Part B)	Paste its content
GitHub release with the APK	Only after *publish release* and the APK path (C20 Part C)	Provide the APK path and approve
Gemini key	Never stores it in files	Enter it in the app only if you run C16
Growth experiment, business case, pitch deck, demo video, team names	Templates and placeholders only	Real content

Claude must never fabricate a deployed URL, an APK, a release, a Lighthouse result, a score or a submission. Placeholders such as LIVE_URL, APK_URL, DEMO_VIDEO_URL, TEAMMATE_1 and TEAMMATE_2 stay until you supply the real values. Never commit `signing.keystore`, `signing-key-info.txt`, `.env` files or keys.

8. Final release and submission checklist

Item	Where to prove it	Produced by or blocked on	Done?
Live link opens on phone and laptop	README LIVE_URL, C19 Part C report	Needs your Vercel account	☐
APK installs on Android	GitHub release (C20)	Needs PWABuilder and your phone	☐
Scorecard: labelled commitments correct, 0 wrong reminders, 0 duplicates	evidence/latest/scorecard.md	Owner/deadline/status needs the fixture (C02, C03)	☐
Starting state saved	evidence/starting_state/	C07 (synthetic until the fixture arrives)	☐
Negotiated scope and final evidence	docs/product-change.md, evidence/final/	C07	☐
Timestamped screenshots	evidence/screenshots/<timestamp>/	C17	☐
Test report	evidence/latest/test-report.txt	C07	☐
Release check all PASS or explained	docs/release-check.md	C18	☐
API documented and live	docs/api.md, <LIVE_URL>/api/health	C11, C19	☐
PWA installable and offline	e2e/pwa.spec.ts result	C12	☐
Growth experiment: ten real rows	docs/growth-experiment.md	You fill the template from C17	☐
Pilot plan with three metrics	docs/pilot-plan.md	C17	☐
Business case PDF and pitch deck	docs/ (your files)	You provide	☐
README with honest limitations	README.md	C17	☐
Commits only under your name, human-style	GitHub Insights > Contributors; git log	Checked in C18	☐
Demo video (about 3 minutes)	README DEMO_VIDEO_URL	You record and link	☐
Official deadline and demo format confirmed	Organisers	Unknown in the brief	☐

9. Audit notes

- **Sources inspected:** the repository code, tests and git history; README.md; docs/integration-status.md, fixture-notes.md, track-a-status.md, track-b-status.md; package.json and lockfile; the C14 brief; the original playbook (C:\Users\shiva\projects\followthru\FollowThru_Prompts_copy_paste.md and 1_FollowThru_Prompt_Playbook.pdf, 40 pages, including its MERGE, FIX, CONTINUE, SCORECARD, deploy, APK and submission pages); GitHub branches and PRs #1-#4.
- **Not found:** the earlier project status and continuation PDF; CLAUDE.md in this repository; the challenge fixture; generated test reports or screenshot folders committed to the repository (e2e screenshots exist only under the ignored test-results/ folder).
- **Design differences from the original plan that are intentional:** commitment status values are open, done, cancelled and deferred, with "overdue" derived from the deadline at the evaluation clock; draft kinds are reminder and clarify, with tone carried by the flow step; human decisions are stored as an audited log and replayed by the engine, instead of persisting resolved commitments; routes are /dashboard, /queue and so on, not /app/*.
- **Drive-letter test failure:** three `npm test` runs during this audit failed with all files reporting "no tests". The cause was isolated: Vitest started with the working directory `d:/followthru` (lowercase drive) fails every suite with "Cannot read properties of undefined (reading 'config')"; the same command from `D:/followthru` passes 148/148. Reproduce with: `node -e "require('child_process').spawnSync('npx vitest run', {cwd:'d:/followthru',shell:true,stdio:'inherit'})"`. The first playbook commit was made right after one such run (documentation only, no code). C13 fixes it.
- **Test counts by area at the start:** deadline 28, time 5, validation 17, CSV/JSON 15, contract 3, engine 44, API 13, store 11, UI 8, app 3, sanity 1 (unit total 148), plus 4 browser tests.